ECE 260C, Spring 2026

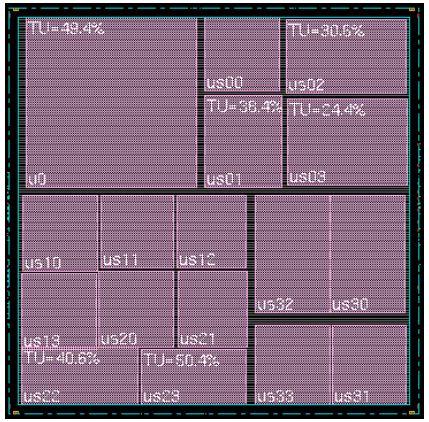
Placement

Andrew B. Kahng

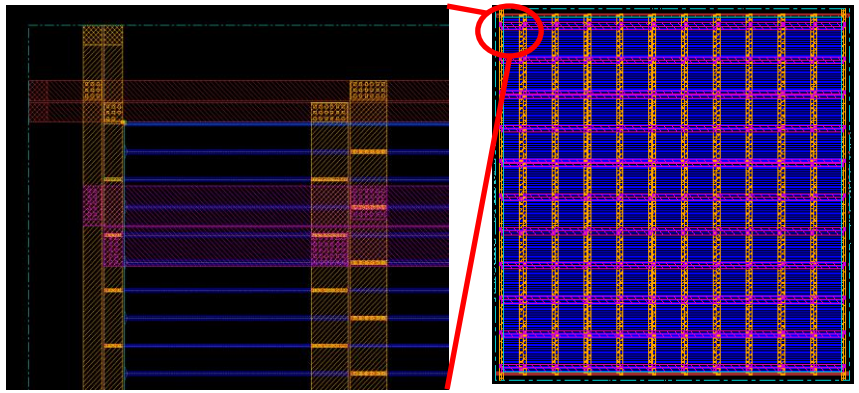
Thanks to: Andrew Kennings, Mingyu Woo, SangGi Do, Nima Darav

Physical Design Flow Pictures (old ECE 260B slide)

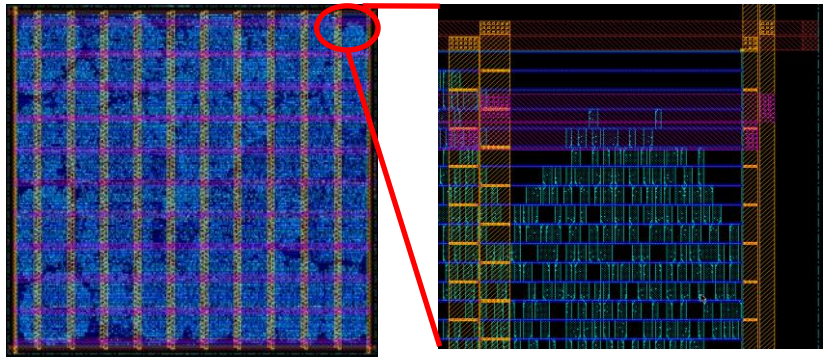
- Floorplanning



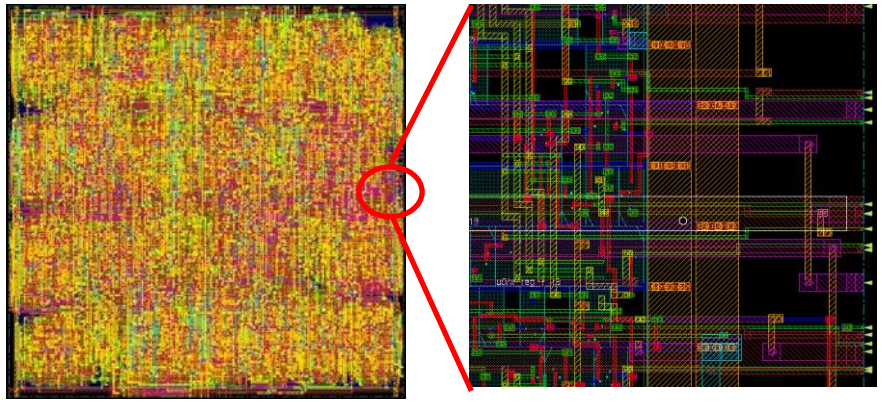
- Powerplanning



- Placement



- Routing



The Placement Problem

Input:

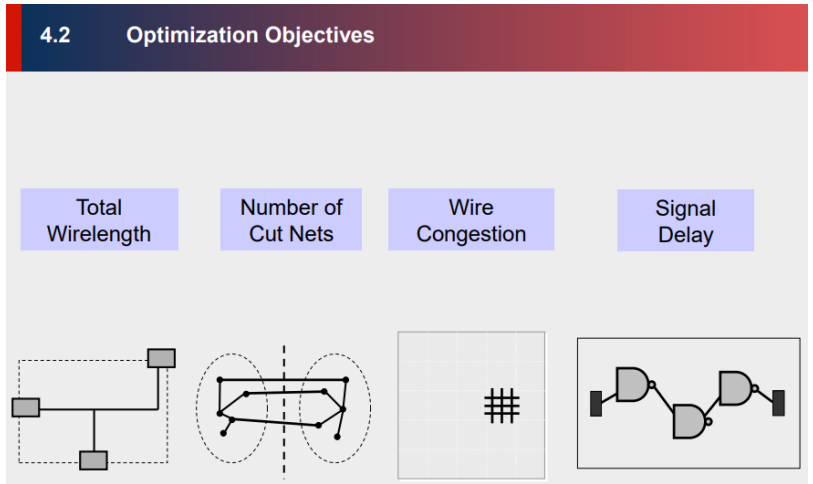
- A set of cells and macros, and their library information
- Connectivity information between cells/macros (and I/Os) (netlist information)
- Fixed layout resource (site map)
- Fixed placements (I/Os, macros); grouping and region (floorplan) constraints
- Activity information, power / clock gating information, …

Output:

- A set of locations on the chip; one location for each cell (x,y,orientation)
- (Legal: non-overlapping, compatible with sites, etc.)

Objective:

- Routability (because the die is fixed)
- Timing and Signal Integrity
- Wirelength
- Power

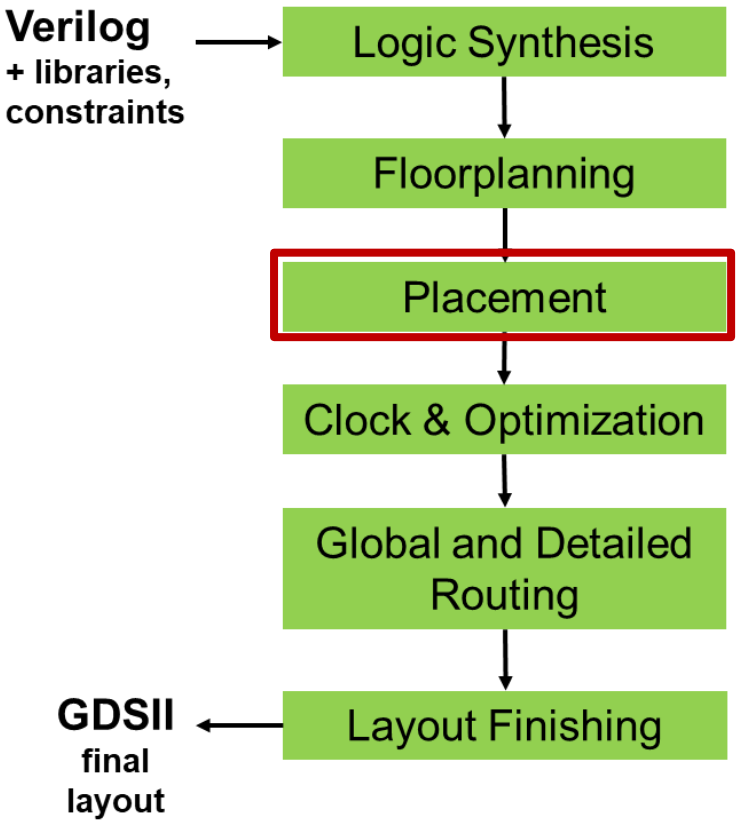


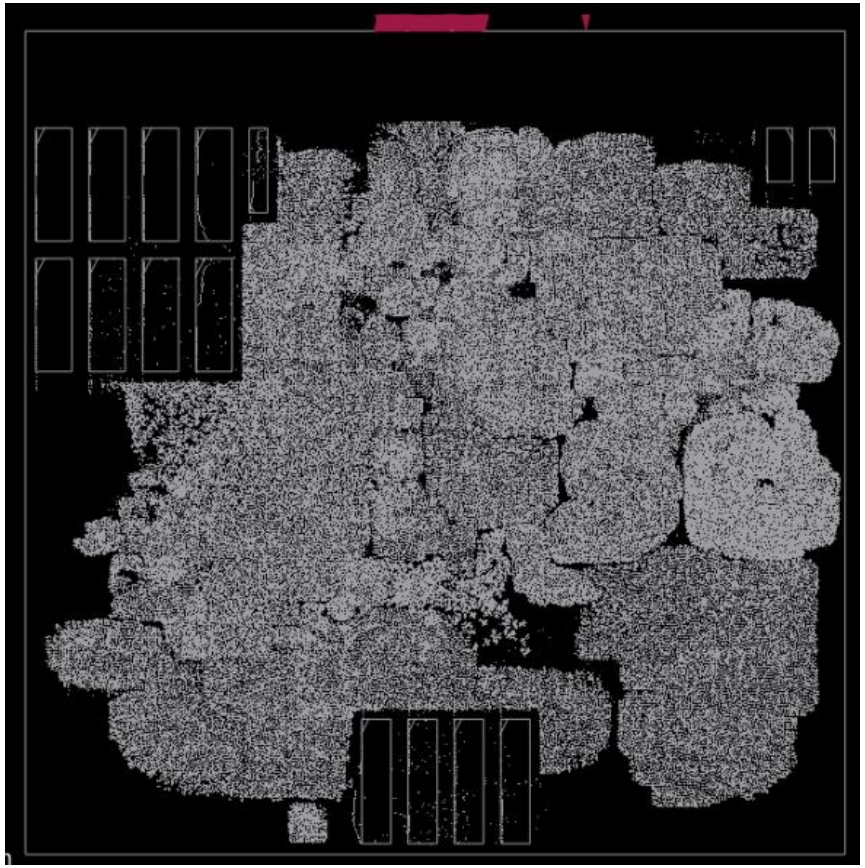
Details:

- Datapath planning; ECO (engineering change order = incremental) placement after resynthesis/sizing; block placement; physical synthesis services; …

GPL

Global Placement





Placement Overview

- ***Placement***
  - Determines the locations of standard cells and/or logic elements while addressing optimization objectives
  - Highly important physical design step in integrated circuit (IC) design flow
    - Directly impacts on timing closure, die utilization, routability, design turnaround time (TAT) → operating frequency, yield, power consumption, cost
- **Placement instances**
  - *Hypergraph* $G = (V, E)$
    - V = a set of vertices, i.e., standard cells and macros
    - E = a set of *hyperedges*, i.e., nets
- **Placement solution** $v = (x, y)$, X- and Y-coordinates of all placeable vertices
  - *legal solution*?

1. Every instance should be settled in the placement region.
2. Every standard cell should be placed within predefined rows.
3. No overlap is allowed between instances including both standard cells and macros.

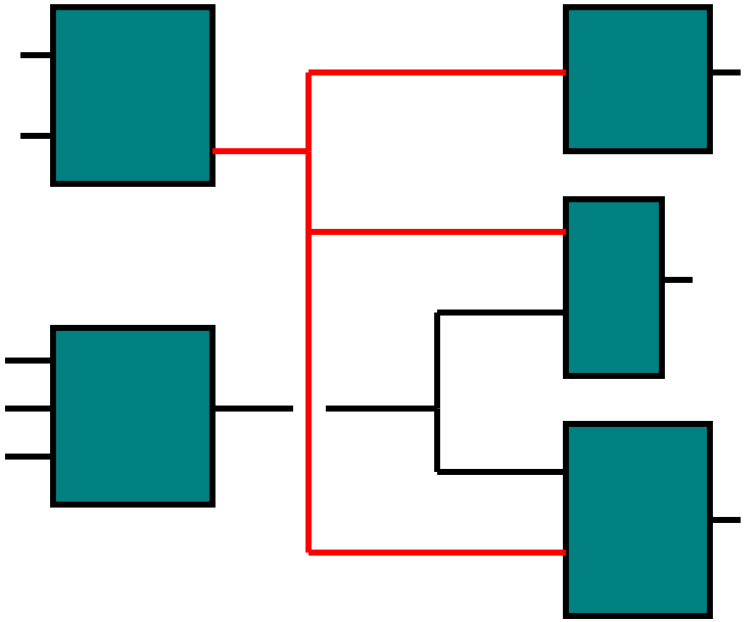
A. B. Kahng, J. Lienig, I. L. Markov and J. Hu, *VLSI Physical Design: From Graph Partitioning to Timing Closure*, Springer, 2011

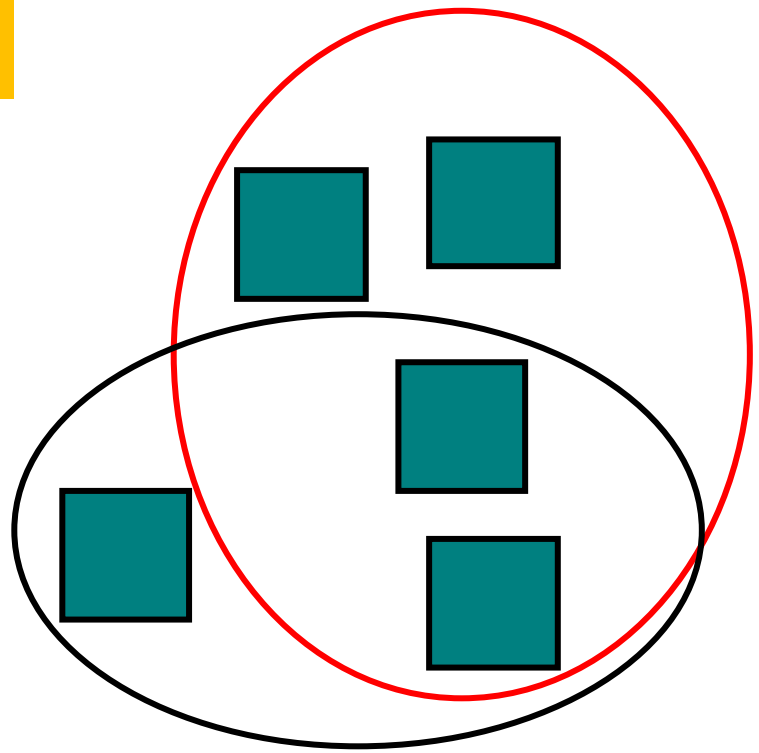
Hypergraphs in VLSI CAD

- Circuit netlist represented by **hypergraph**

Graph G = (V, E_G)
An edge in E_G is a subset of V with cardinality = 2

Hypergraph H = (V, E_H)
A hyperedge in E_H is a subset of V with cardinality ≥ 2

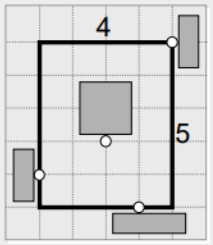
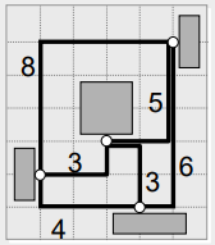
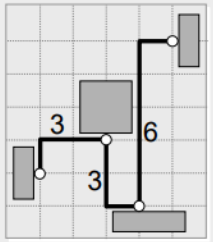
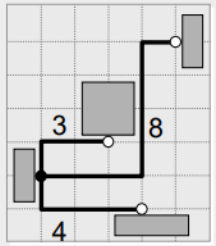




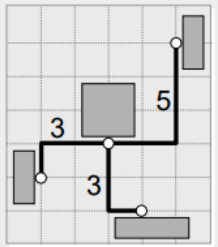
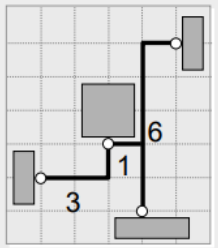
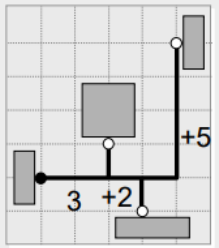
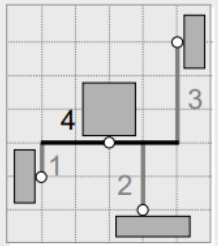
Example: **5** vertices, **2** hyperedges (one of size 4, and one of size 3)

How Should the Placer See a Netlist? (KLMH book slides)

Wirelength estimation for a given placement

Half-perimeter wirelength (HPWL)	Complete graph (clique)	Monotone chain	Star model
			
HPWL = 9	Clique Length = $(2/p)\Sigma_{e \in \text{clique}} d_M(e)$ = 14.5	Chain Length = 12	Star Length = 15

Wirelength estimation for a given placement (cont'd.)

Rectilinear minimum spanning tree (RMST)	Rectilinear Steiner minimum tree (RSMT)	Rectilinear Steiner arborescence model (RSA)	Single-trunk Steiner tree (STST)
			
RMST Length = 11	RSMT Length = 10	RSA Length = 10	STST Length = 10

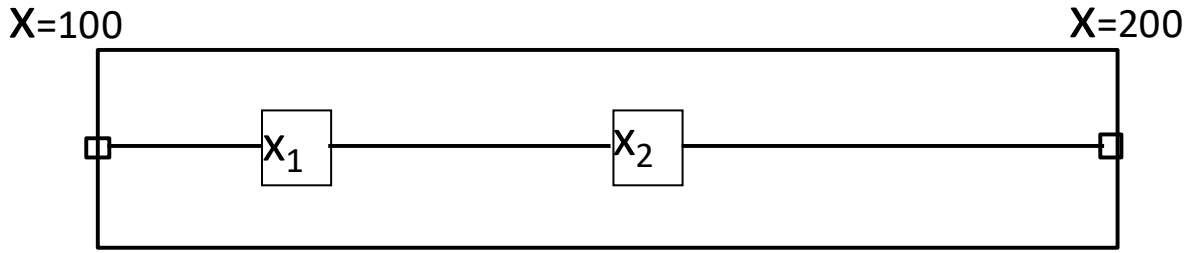
One Key: How to Estimate Wirelength / Model Nets

- Star topology (two types, second w/extra vertex being Hu-Moerder 1985)
  - With 'overlaps', called an "arborescence" (Rao, Sadayappan, Shor ~1992)

- Half-perimeter of bounding box (BB) of terminal locations
  - "HPWL"
  - Exact for #pins p = 2, p = 3
  - Fixes for p > 3, BBox aspect ratio: Cheng ICCAD94, Caldwell et al. ISPD98
    - *On Wirelength Estimations for Row-Based Placement*

- *Issues:
  - Timing-driven? (see: "shallow-light tree", "AHHK" or "Prim-Dijkstra" tree, etc.)
    - *An Efficient Timing-Driven Global Routing Algorithm*
    - *Prim-Dijkstra Tradeoffs for Improved Performance-Driven Routing Tree Design*
    - *Prim-Dijkstra Revisited: Achieving Superior Timing-driven Routing Trees*
  - Congestion? How to account for detouring (which causes more detouring)
  - Bend minimization vs. via stacking, stability, …
  - Post-signal integrity optimizations (spreading)? Details of track blockage?

“Quadratic Placement” Example



$$Cost = (x_1 - 100)^2 + (x_1 - x_2)^2 + (x_2 - 200)^2$$

$$\frac{\partial}{\partial x_1} Cost = 2(x_1 - 100) + 2(x_1 - x_2)$$

$$\frac{\partial}{\partial x_2} Cost = -2(x_1 - x_2) + 2(x_2 - 200)$$

Setting the partial derivatives = 0, we solve for the minimum Cost:

$$Ax = b$$

$$\begin{bmatrix} 4 & -2 \\ -2 & 4 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 200 \\ 400 \end{bmatrix}$$

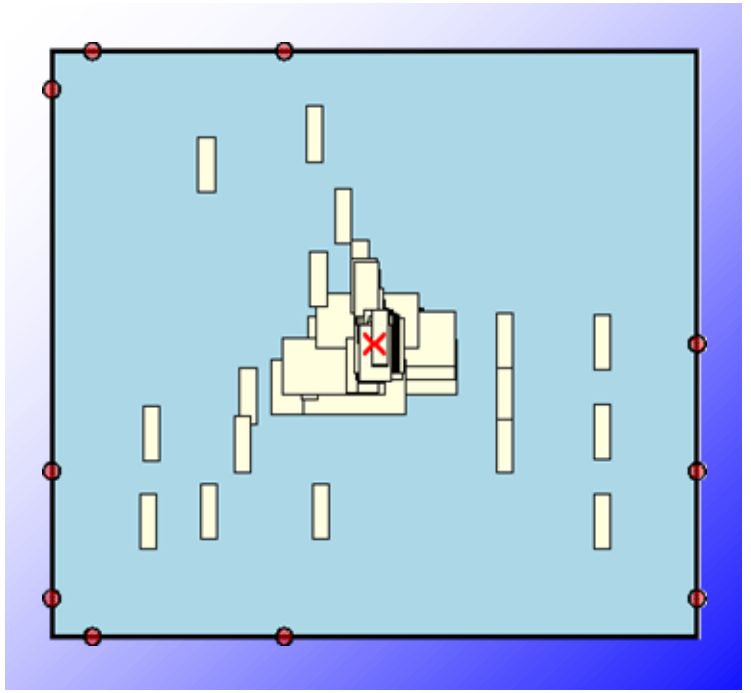
$$\begin{bmatrix} 2 & -1 \\ -1 & 2 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 100 \\ 200 \end{bmatrix}$$

$$x_1 = \frac{400}{3} \qquad x_2 = \frac{500}{3}$$

A_{ii} = degree of a node
A_{ij} = -(i-j) connectivity
b_i = sum of locations connected to cell i

Why “Squared Wirelength” and “Quadratic Placement”?

- Because we can
- Because it is trivial to solve
- Because there is only one solution
- Because the solution is a global optimum
- Because the solution conveys “relative order” information
- (Because the solution conveys “global position” information)
- **Key issue is “spreading”**
  - **Need “anchors” to begin with, else the optimal placement is trivial / vacuous**
  - **There is a tension between “make wires short” (remember, we may not even have estimated the wires very well) and “make sure that the placed objects don’t overlap”**
  - **Also:** routability? timing? power? legal sites? … mixed-size or macros-before-cells? … … … …



B2B ("Bound-to-Bound") Net Model: Why it was Proposed

- Quadratic placers need a two-pin net model inside a quadratic objective.
- Traditional clique modeling uses all pin pairs, but only approximates HPWL.
- Kraftwerk's B2B model targets exact HPWL representation in the quadratic cost.
- HPWL is attractive because it is a fast proxy for routed wirelength.
- The model also aims to reduce overlap side-effects caused by clique inner-pin connections.
- *Kraftwerk: A Fast and Robust Quadratic Placer Using an Exact Linear Net Model* = figure source
- **The** seminal paper: Eisenmann and Johannes, DAC-1998. *(submission #182?)*

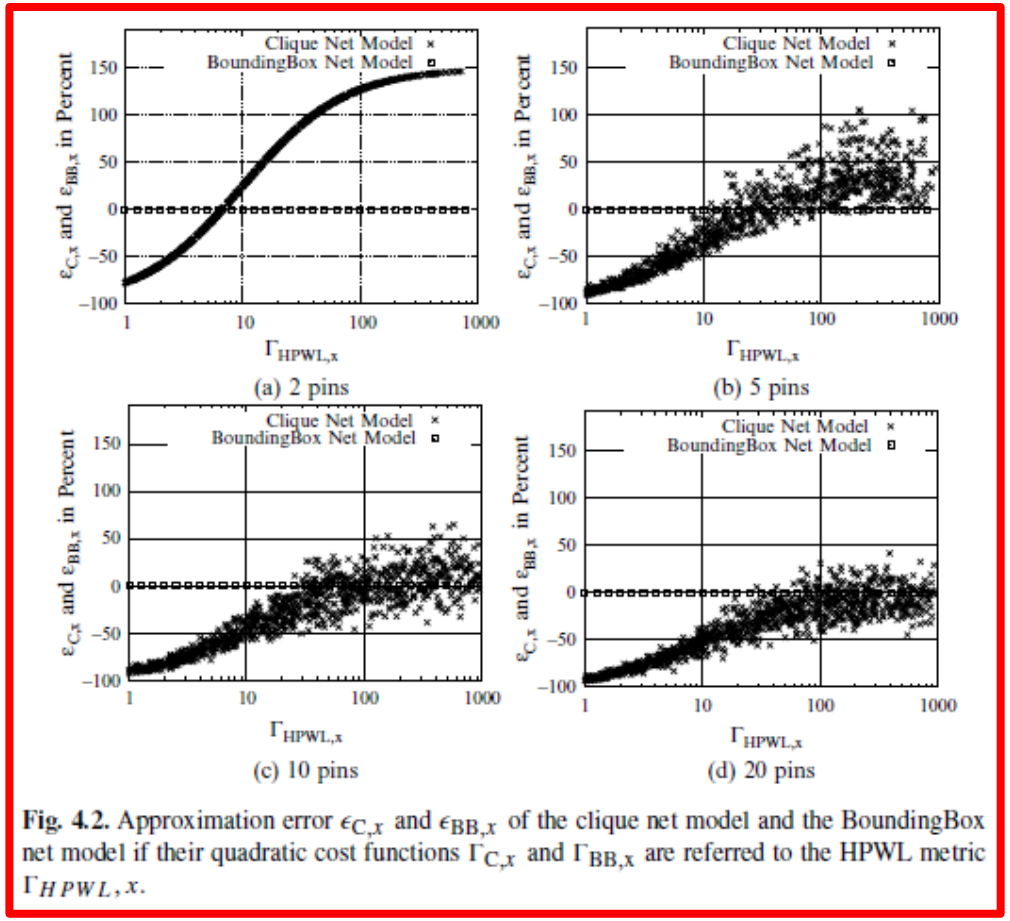


Fig. 4.2. Approximation error $\epsilon_{C,x}$ and $\epsilon_{BB,x}$ of the clique net model and the BoundingBox net model if their quadratic cost functions $\Gamma_{C,x}$ and $\Gamma_{BB,x}$ are referred to the HPWL metric $\Gamma_{HPWL,x}$.

Approximation error of clique vs. BoundingBox model (errors for 2, 5, 10 and 20 pins)

B2B Construction in 1-D

- Pick the left bound pin (a) and right bound pin (b).
- Add one connection from (a) to (b), with length equal to bounding-box width (w).For each interior pin, connect it to both bounds.
- Number of connections becomes (1 + 2(P-2)), which is linear in pin count.
- Use weights $w_{x,i} = (2 / (P - 1)) * (1 / l_{x,i})$

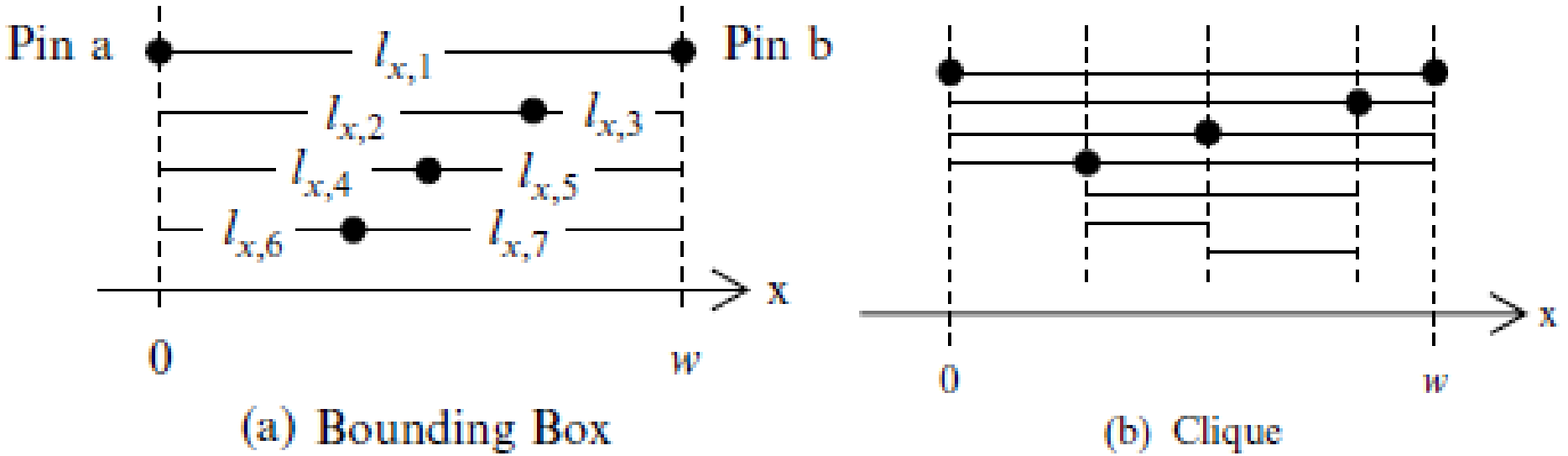


Fig. 4.3. BoundingBox and clique net model of a five pin net in x-direction.

Benefits from B2B

- In x-direction, the quadratic cost equals the net bounding-box width (w)
- Similarly in y-direction, so total B2B cost equals HPWL = w + h
- Connection count grows **linearly** with pin count, unlike the clique model's quadratic growth
- Fewer inner-pin edges means less artificial "gluing" of inner pins
- In Kraftwerk authors' experiments, BoundingBox improved HPWL by about 9.93% and CPU by about 30.4% vs. clique

Table 4.2. The BoundingBox net model compared to the clique net model based on the eight circuits of the ISPD 2005 contest benchmarks.

	Clique Net Model		BoundingBox Net Model	
Circuit	HPWL [m]	CPU [s]	HPWL [m]	CPU [s]
adaptec1	90.11	374	82.67	256
adaptec2	101.55	460	93.03	344
adaptec3	251.88	641	229.36	687
adaptec4	218.66	829	200.85	714
bigblue1	108.79	679	97.97	505
bigblue2	171.21	1070	155.43	551
bigblue3	386.21	4078	344.94	2072
bigblue4	961.85	9682	859.18	4180
Average	1.000	1.000	0.907	0.696
Improvement to Clique			**9.93%**	**30.4%**

Improving the Quadratic Objective Function in Module Placement *

Lars Hagen and Andrew B. Kahng

UCLA Department of Computer Science
Los Angeles, CA 90024-1596

ASIC-1992 paper -- looking at this ~35 years ago …

Wot the L: Analysis of Real versus Random Placed Nets, and Implications for Steiner Tree Heuristics *

Andrew B. Kahng[1,2], Christopher Moyes[1], Sriram Venkatesh[1] and Lutong Wang[2]
[1]CSE and [2]ECE Departments, UC San Diego, La Jolla, CA 92093
{abk, cmoyes, srvenkat, luw002}@ucsd.edu

ISPD-2018 paper on what HPWL objective implies for the placed pin locations of signal nets!

RePlAce: Advancing Solution Quality and Routability Validation in Global Placement

Chung-Kuan Cheng, Andrew B. Kahng, Ilgweon Kang and Lutong Wang

(based on slides from Ilgweon Kang's Ph.D. defense)

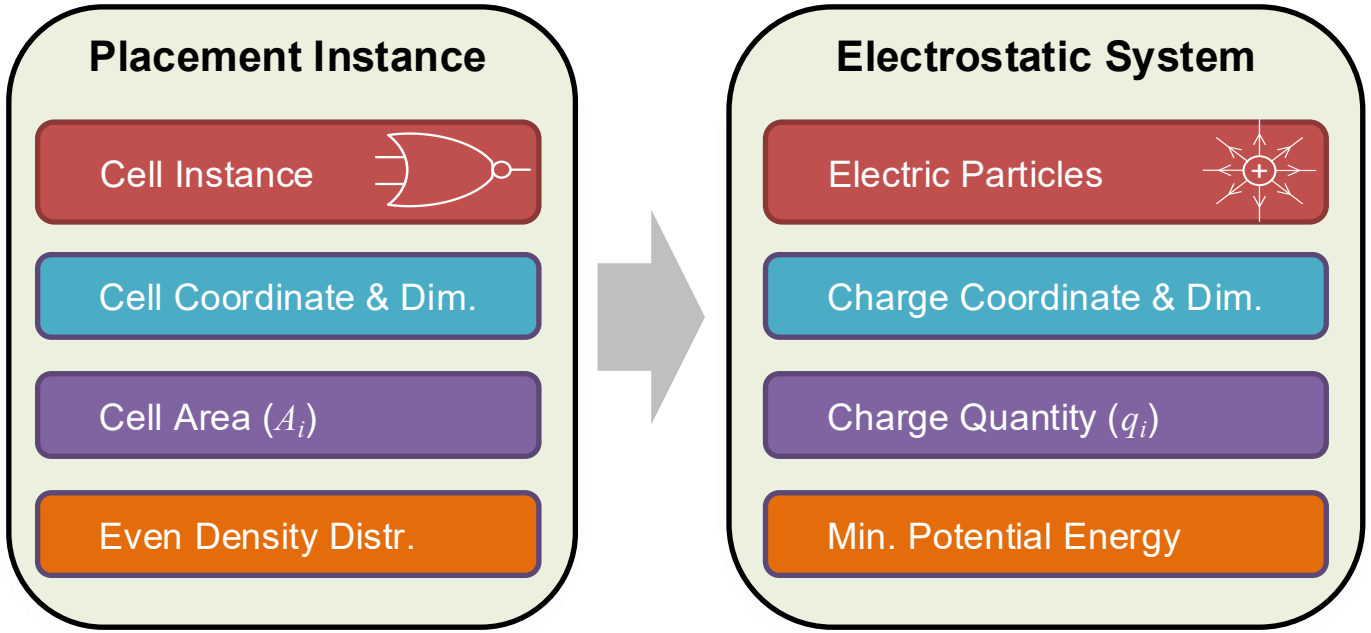
paper

Problem Formulation

- Placement Objective Function $f(v)$

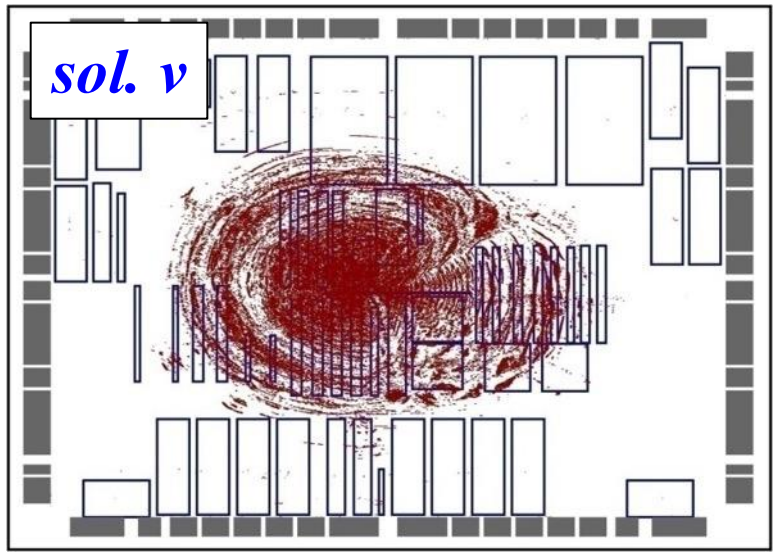
$$\min_{\boldsymbol{v}} f(\boldsymbol{x}, \boldsymbol{y}) = W(x, \boldsymbol{y}) + \sum_{b} \mathcal{V}_b \, D_b(x, y)$$

- ePlace: Electrostatics-based global-smooth density function

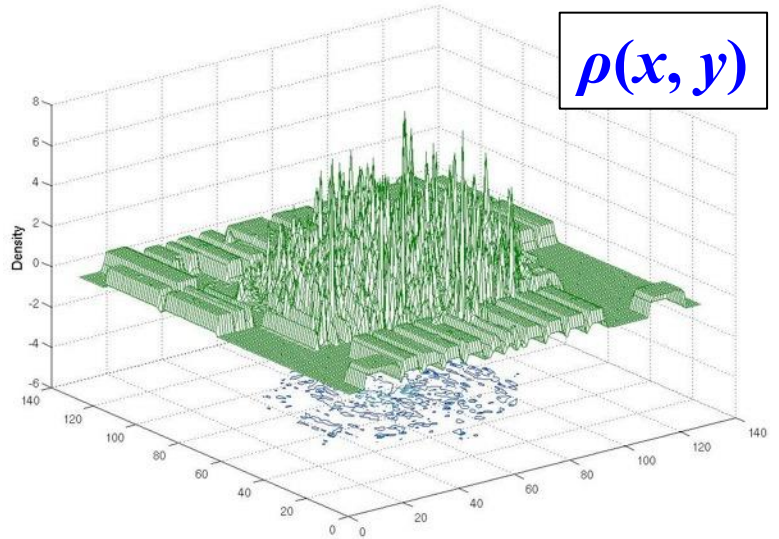


[13] J. Lu, "Analytic VLSI Placement Using Electrostatic Analogy", Ph.D. Dissertation, UC San Diego, CA, 2014.

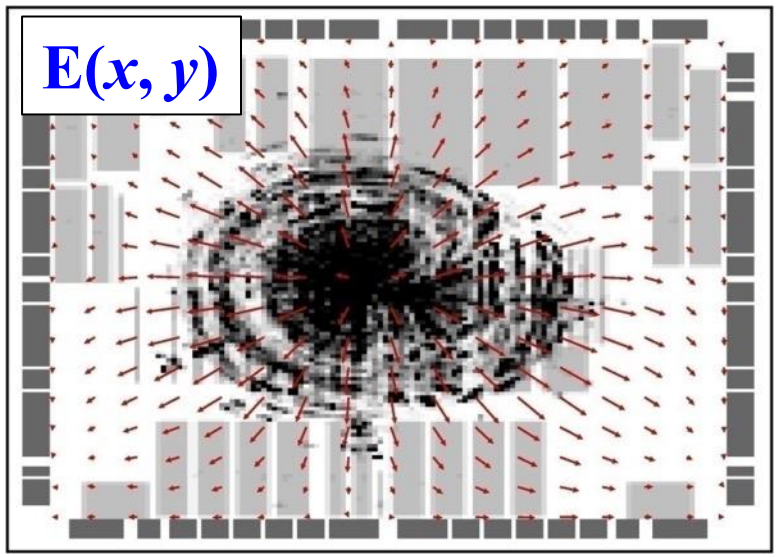
ePlace: In Each Iteration



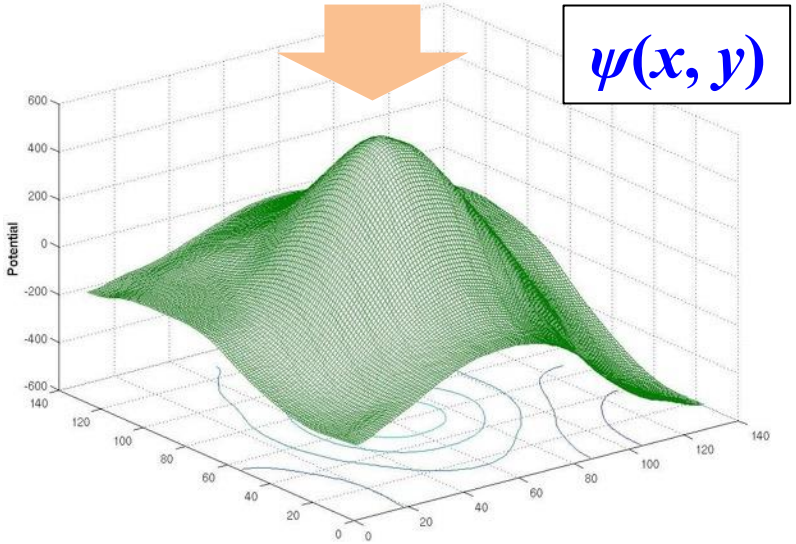
cell & macro distr.



charge density distr.



electric field distr.



electric potential distr.

J. Lu, "Analytic VLSI Placement Using Electrostatic Analogy", Ph.D. Dissertation, UC San Diego, CA, 2014.

RePlAce

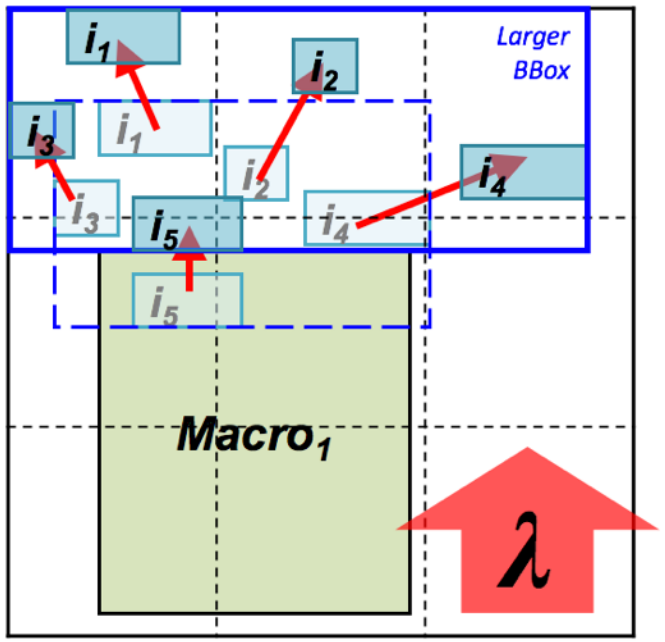
- Unified placement engine which solves multiple classes of academic benchmarks (mixed-size, fixed-macros, routability-driven, etc.
- Local Lagrange multiplier
  - Enables local smoothing w/awareness of local over-demanded bins
- Meta parameter tuning
  - Adjust step size of numerical method
- Routability optimization
  - Simple but effective metal layer-aware superlinear cell inflation
- Differences from the previous ePlace 2.0
  - Routability optimization techniques
  - Code optimizations for ~4X speedups
  - Various improvements to placement mechanisms
    - E.g., macro legalization using annealing; amount of rollback after fixing macro; handling of overflows; bin size determination and local smoothing; …

RePlAce: Constraint-Driven Placement

Local Lagrangian multiplier v_j

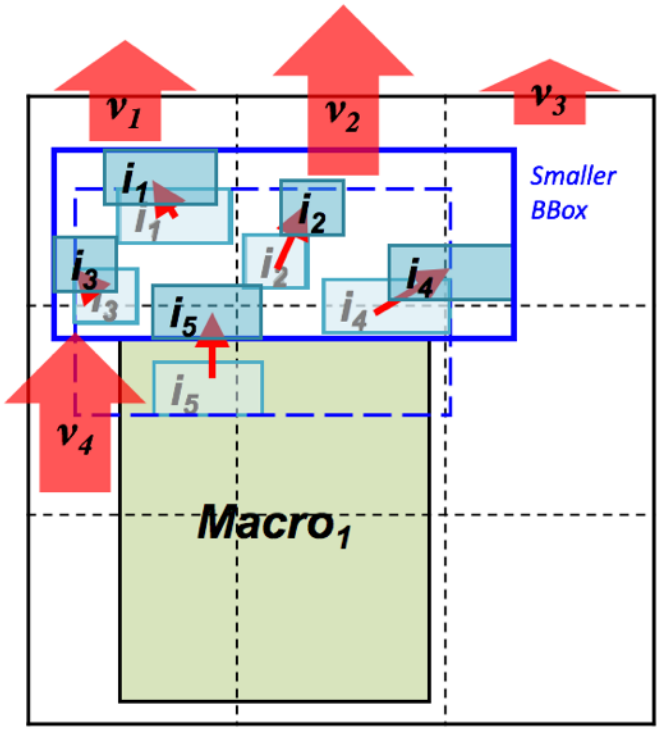
$$v_j = e^{\alpha(BinDemand_j - BinCapacity_j)}$$

- Local-Density Penalty Factor for each bin b_j
- $BinDemand_j$ = total cell area in bin b_j
- $BinCapacity_j$ = area of bin b_j
- $BinDemand_j$ - $BinCapacity_j$ = $overflow_j$ of bin b_j.



Global λ ↑, and Larger HPWL ↑

[ePlace]



Local v_i ↑, and Smaller HPWL ↑

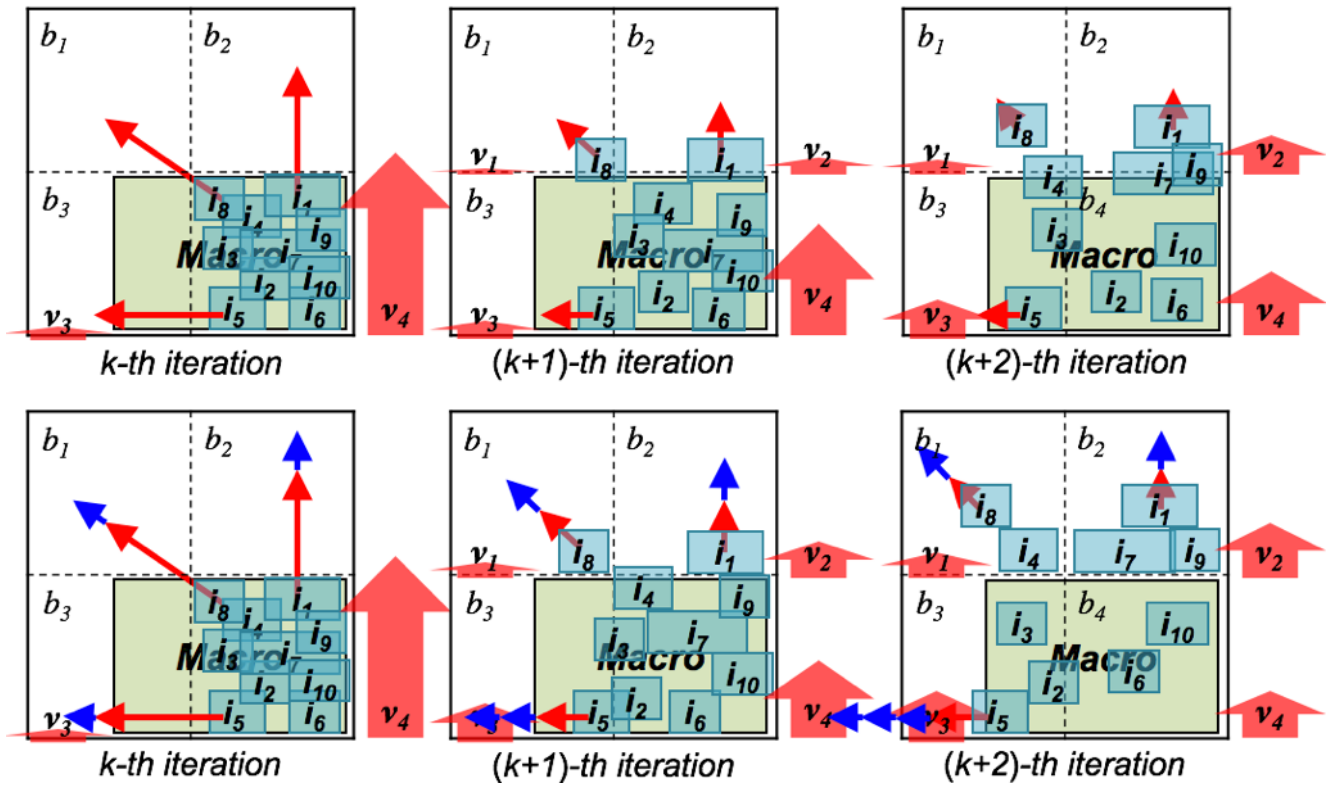
[RePlAce with Constraint-Driven]

RePlAce: Constraint-Driven Placement

- Local-Density Cost Coefficient for each cell

$$\Delta_i^{iter+1} = \Delta_i^{iter} + \beta \cdot \frac{\max(Overflow_j, 0)}{\sum_i A_i} \qquad i \square b_j.$$

  - The previously defined v_j was insufficient for local smoothing
  - Cell i from high-overflow bin loses a large amount of local-oriented repulsive force
  - Δi helps to memorizes previously obtained local force.

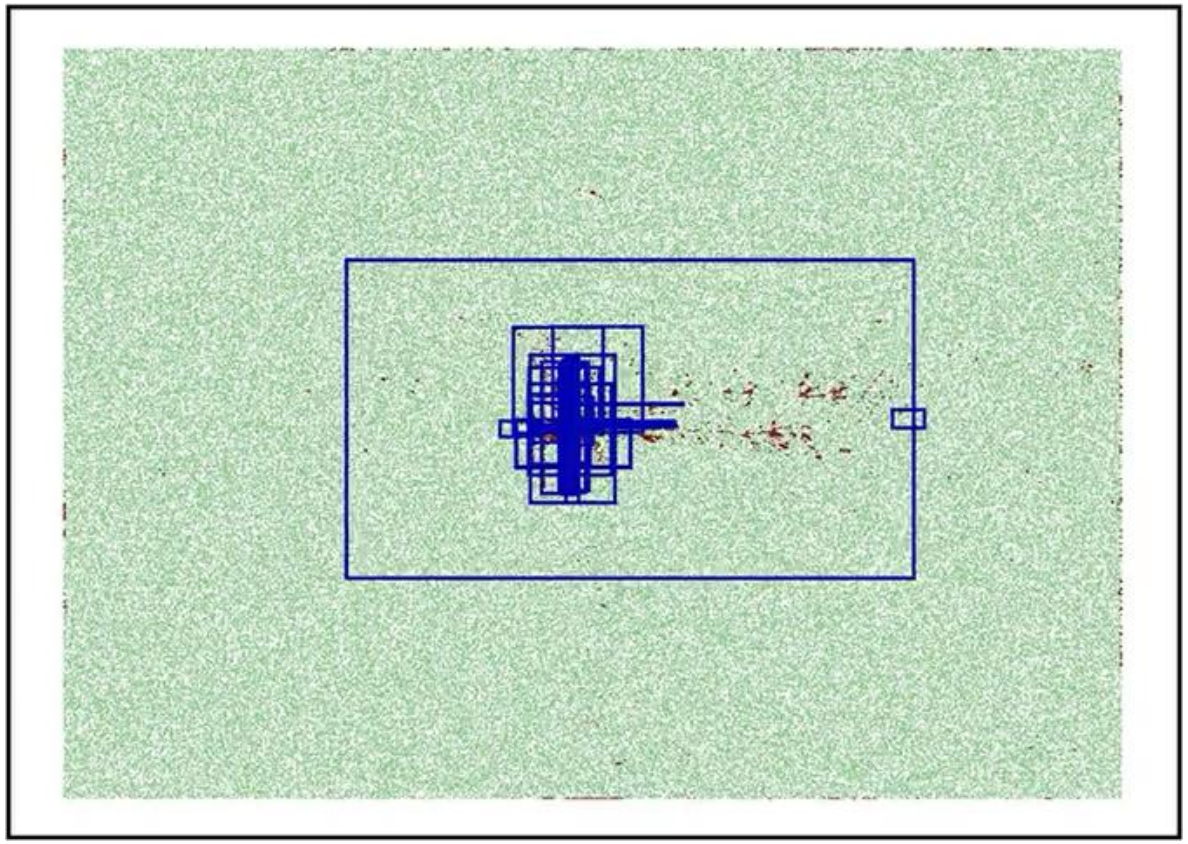


Local smoothing **without** local-density cost coefficient Δ_i

Local smoothing **with** local-density cost coefficient Δ_i

RePlAce: Constraint-Driven Placement

Global placement animation with local Lagrangian multiplier



NEWBLUE1, [RePlAce with local Lagrangian multiplier, i.e., constraint-driven placement]

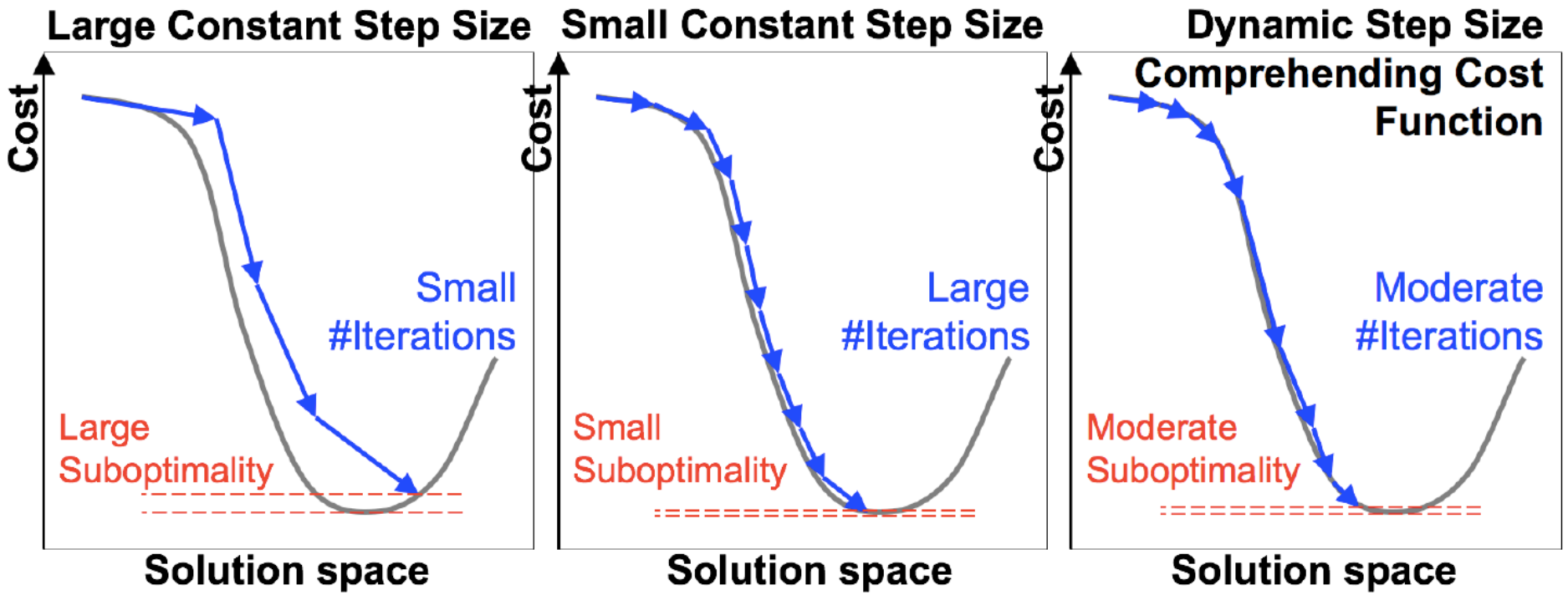
HPWL = 5.60E+7, #Iter = 762 (623+139), runtime = 9.9 min, target density = 100%

Comparison: ePlace 2.0 with local Lagrangian multiplier

HPWL = 5.71E+7, #Iter = 1078 (609 + 469), runtime = 42 min, target density = 100%

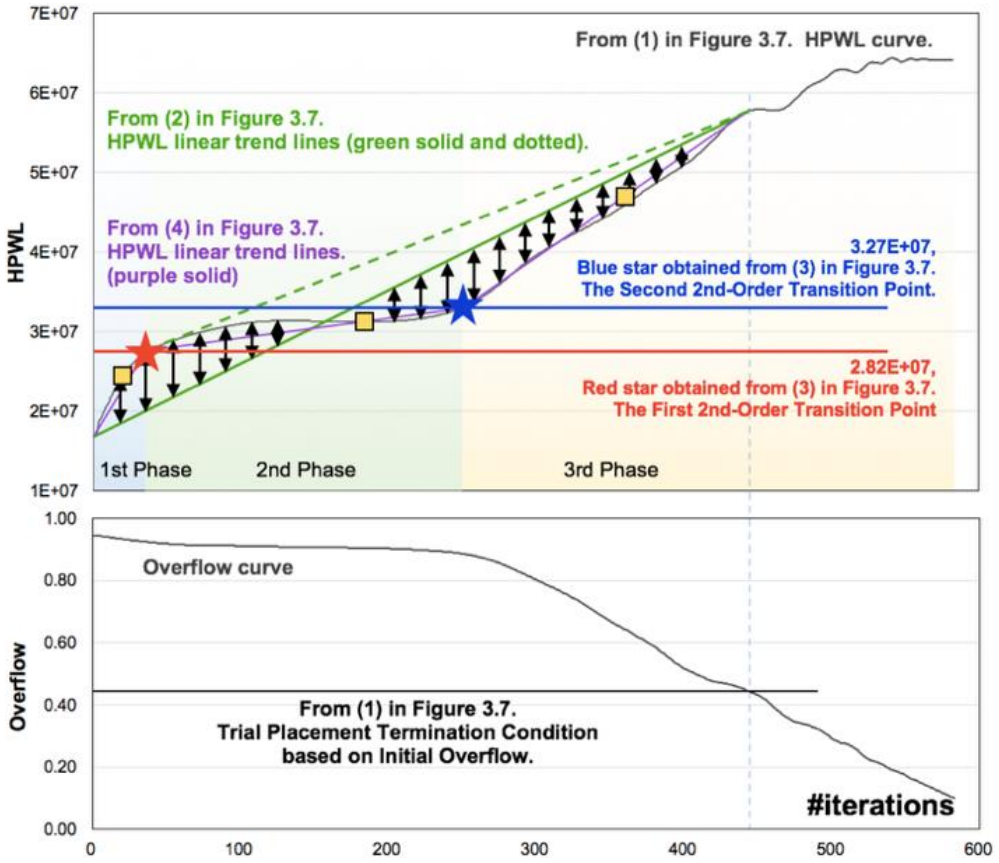
RePlAce: Dynamic Step Size

- General Idea of Dynamic Step Size

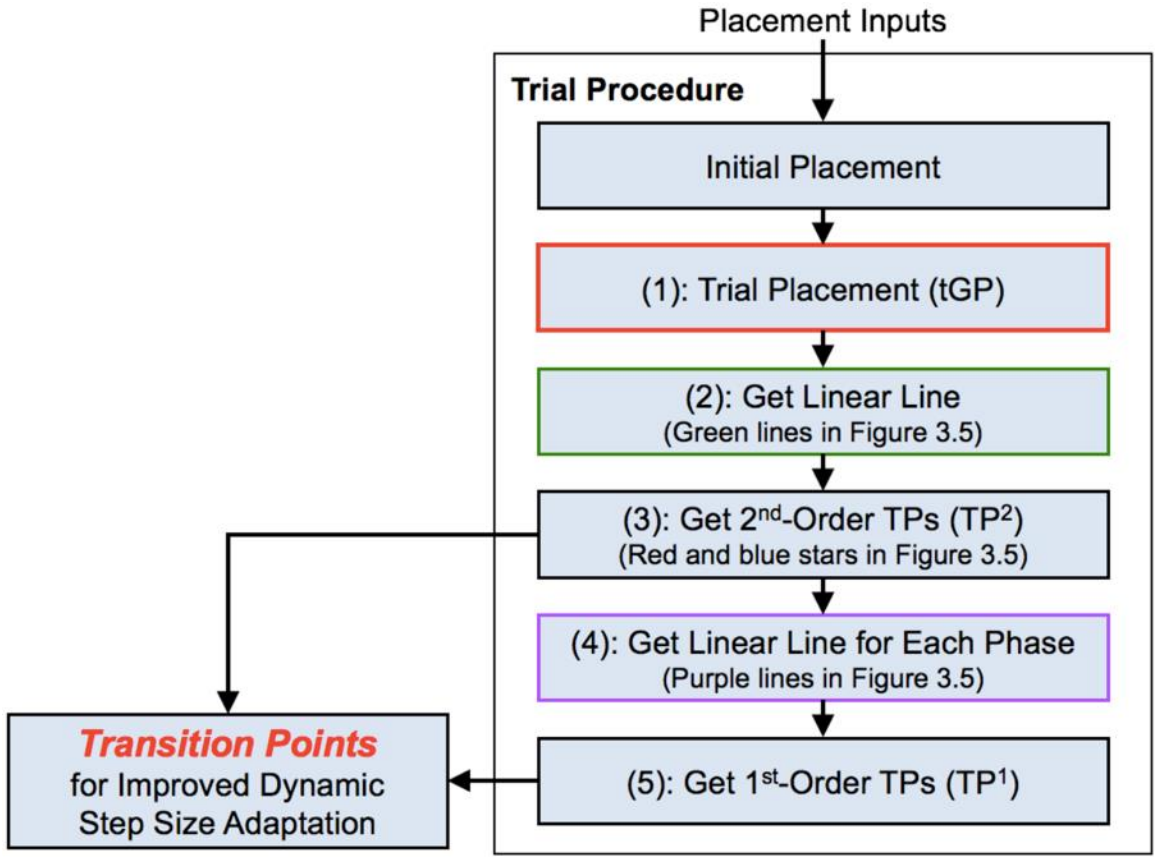


RePlAce: Improved Dynamic Step Size

- Methodology to capture “transition points” on HPWL curve.



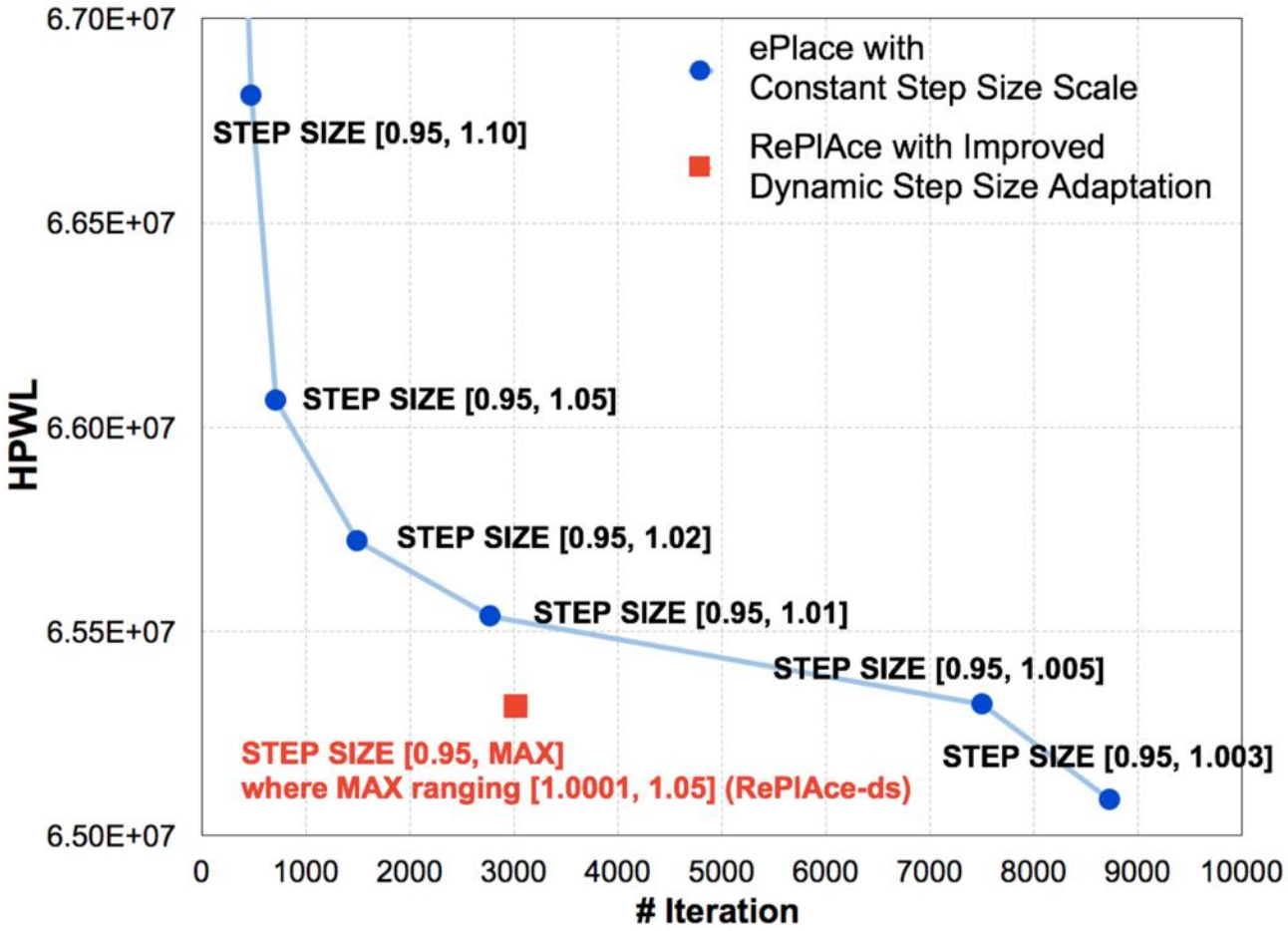
▲ HPWL curve of ADAPTEC1 (MMS) from the trial placement procedure and the estimated transition points (TPs)



▲ Flowchart of trial placement procedure. The red rectangle indicates nonlinear optimization using Nesterov’s method. The actual placement procedure follows this tGP procedure.

RePlAce: Improved Dynamic Step Size

- Solution quality in terms of the final HPWL
  - ePlace vs. RePlAce-ds (ADAPTEC1)
    - RePlAce-ds achieves a dominating runtime and solution quality (red square)
    - Below [0.95, 1.003], e.g., [0.95, 1.002], [0.95, 1.001], etc., runs did not converge



RePlAce: Routability-Driven Placement

- Simple but effective routability optimization
  - A layer-aware cell inflation technique
  - Integrate the official global router NCTU-GR [17] of the DAC-2012 [18] and ICCAD-2012 [19] benchmark suites for congestion estimation
  - Superlinear cell inflation technique to mitigate global routing congestion during global placement
  - We further include a post-placement optimization by [20]
    - Following the strategy of recent leading works [21] [22]

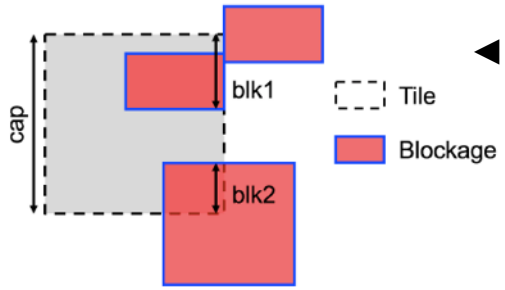
[17] NCTU-GR, http://people.cs.nctu.edu.tw/~whliu/NCTU-GR.htm
[18] N. Viswanathan, C. J. Alpert, C. N. Sze, Z. Li and Y. Wei, "The DAC 2012 Routability-driven Placement Contest and Benchmark Suite", *Proc. DAC*, 2012, pp. 774-782.
[19] N. Viswanathan, C. J. Alpert, C. N. Sze, Z. Li and Y. Wei, "ICCAD-2012 CAD Contest in Design Hierarchy Aware Routability-Driven Placement and Benchmark Suite", *Proc. ICCAD*, 2012, pp. 345-348.
[20] W.-H. Liu, C.-K. Koh and Y.-L. Li, "Optimization of Placement Solutions for Routability", *Proc. DAC*, 2013, pp. 1-9.
[21] X. He, T. Huang, L. Xiao, H. Tian and E. F. Y. Young, "Ripple: A Robust and Effective Routability-Driven Placer", *IEEE Trans. on CAD* 32(10) (2013), pp. 1546-1556.
[22] X. He, Y. Wang, Y. Guo and E. F. Y. Young, "Ripple 2.0: Improved Movement of Cells in Routability-Driven Placement", *ACM Trans. on DAES* 22(1) (2016), pp. 10:1-10:26.

RePlAce: Routability-Driven Placement

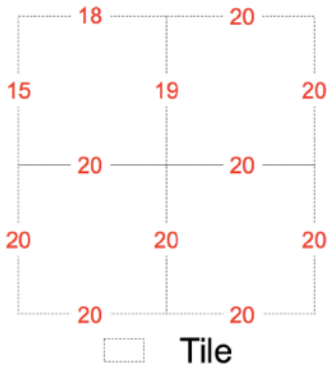
- Metal layer-aware superlinear cell inflation

$$infl\_ratio = \max_{all\ e,ml} \left(\left(\frac{demand_{e,ml} + blk_{e,ml}}{cap_{e,ml}} \right)^{\gamma_{super}}, 2.5 \right)$$

  - e = one of the four edges of a given global routing tile
  - ml = a specific metal layer
  - γ_{super} = 2.33 (empirically determined)



◀ Blockage calculation. For the vertical edge on the right, $blk = blk1 + blk2$. Note the union of blocked capacity for the upper two blockages.

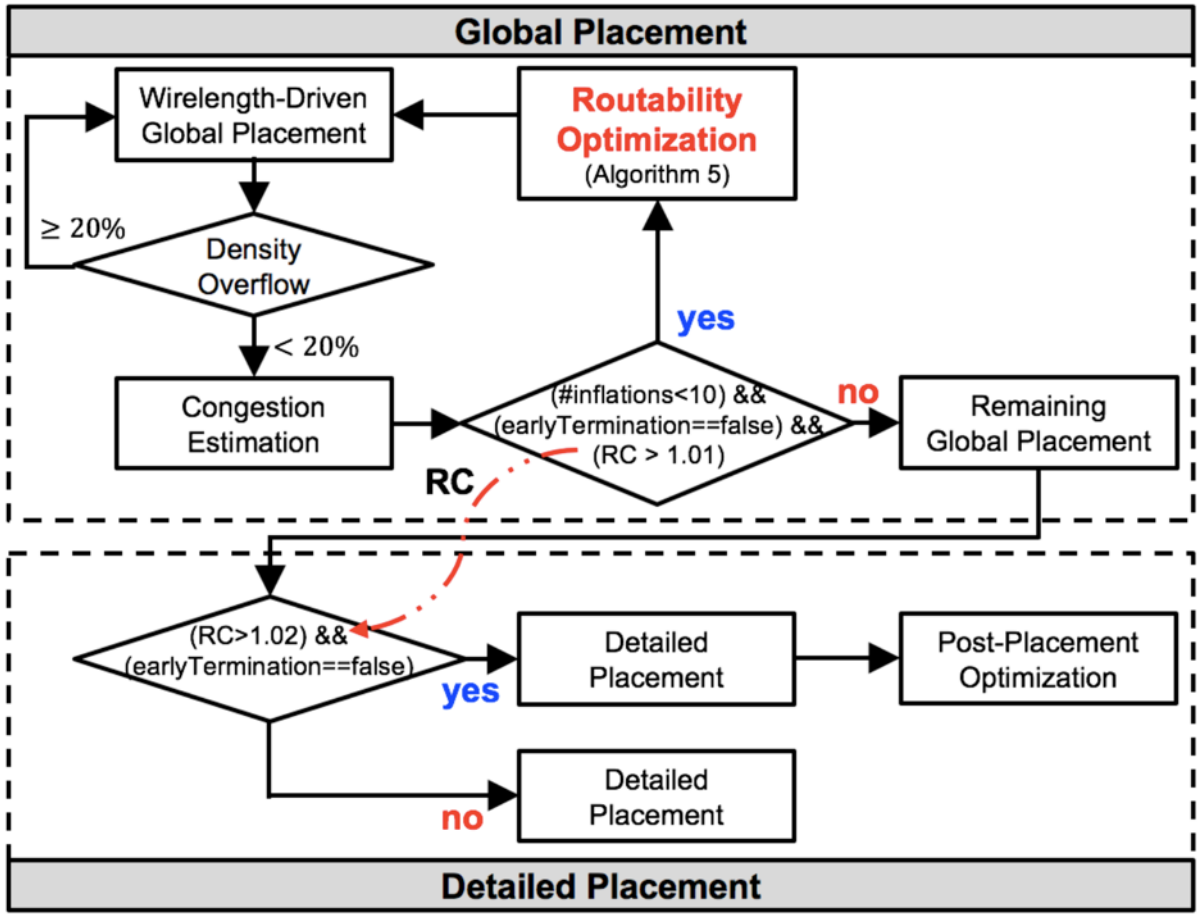


◀ Routing demand calculation: the upper-left tile has a horizontal routing demand of $\max(15,19) = 19$

  - Considers the total available whitespace
    - Starting from 90% die utilization,
    - We limit 'the maximum cell-inflated area' ≤ 10%
    - If exceed, then perform 'inflation ratio adjustment'
      - Divides the inflation ratio for each tile by the inflation ratio of the least-congested tile that has a ratio greater than one

RePlAce: Routability-Driven Placement

- IEEE Trans. on CAD paper: routability optimization flow

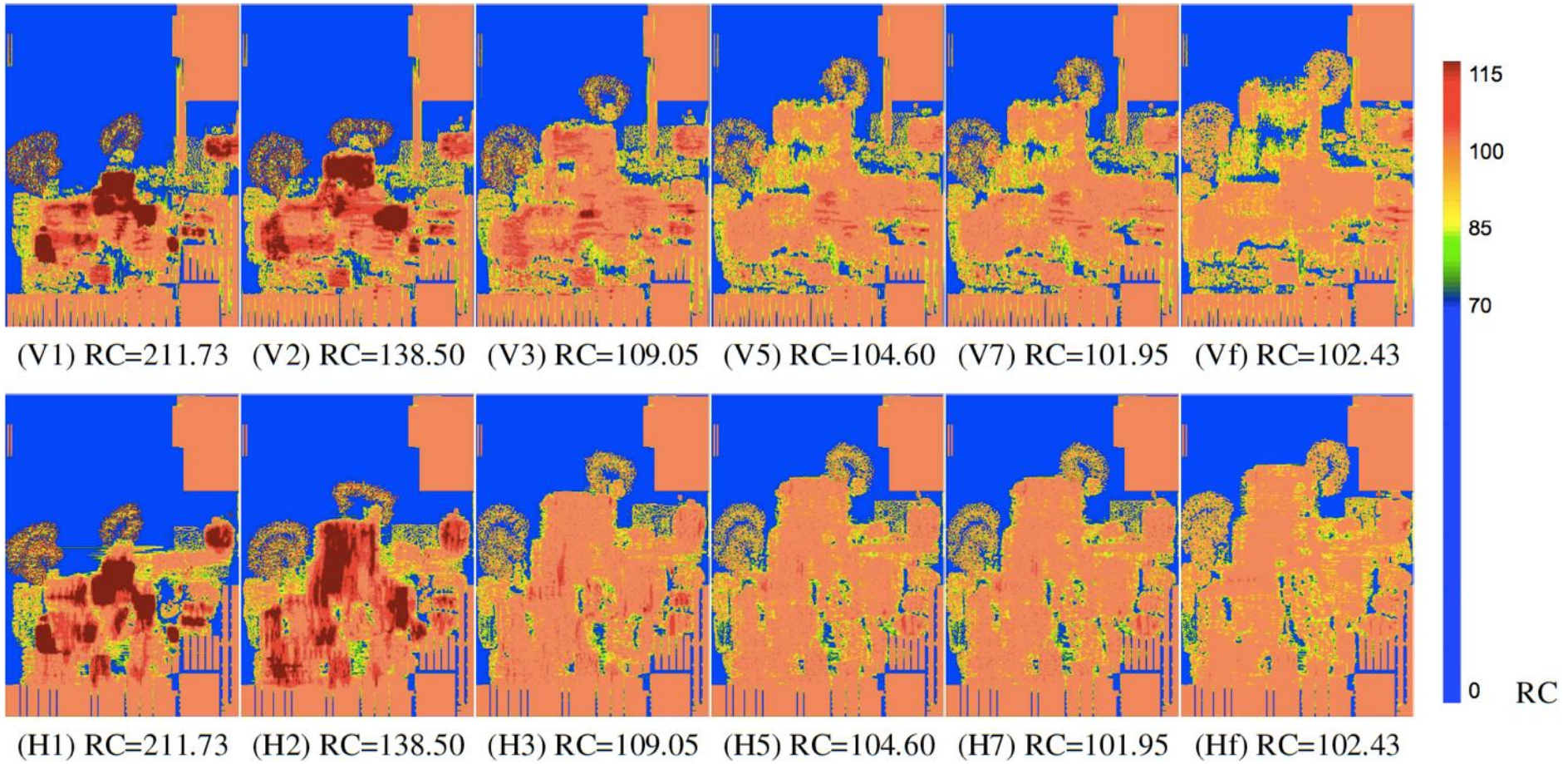


- Uses the detailed placer from NTUplace3

T.-C. Chen, Z.-W. Jiang, T.-C. Hsu, H.-C. Chen and Y.-W. Chang, "NTUplace3: An Analytical Placer for Large-Scale Mixed-Size Designs with Preplaced Blocks and Density Constraints", *IEEE Trans. on CAD* 27(7) (2008), pp. 1228-1240.

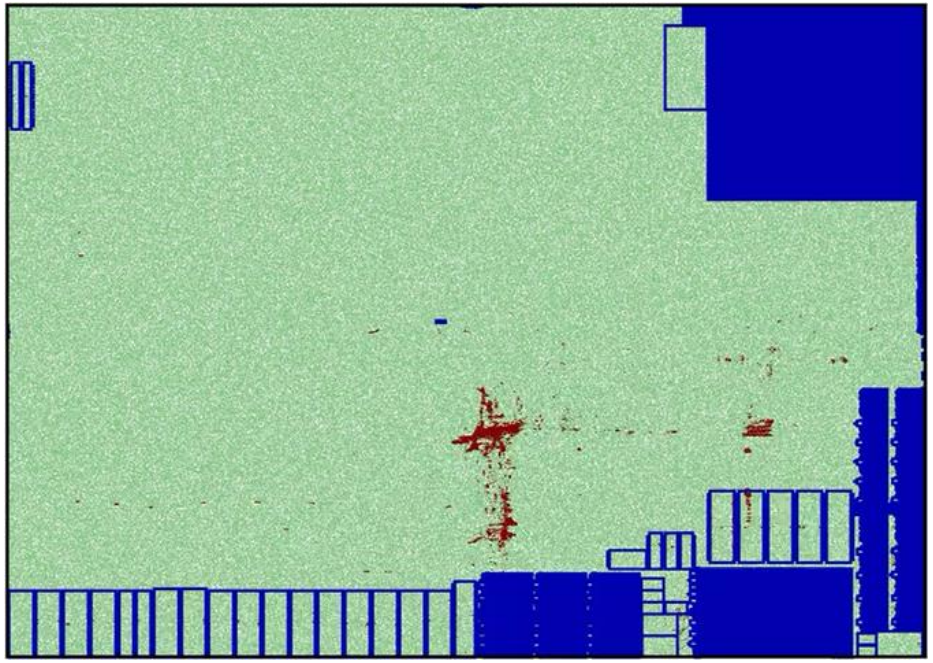
RePlAce: Routability-Driven Placement

- Global routing overflow (SUPERBLUE12) during routability-driven global placement procedure

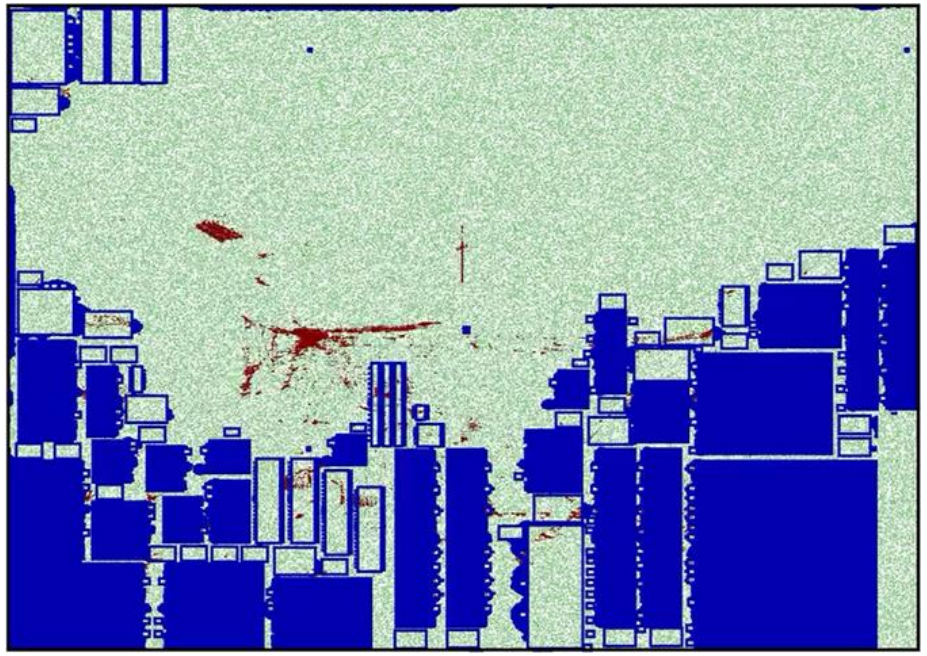


RePlAce: Routability-Driven Placement

- Global placement animation with our routability optimization
  - Keeps the inflated cell size by the end of global placement
  - Red-cell zone is keep growing along with the routability optimization



SUPERBLUE12 (DAC-2012)
The initial RC value = 211.73
The final RC value = 102.43



SUPERBLUE18 (ICCAD-2012)
The initial RC value = 135.23
The final RC value = 102.10

RePlAce (2019) Summary

- Advancing solution quality and routability validation in GPL
  - Local density function
    - Local smoothing comprehending local over-demanded bins
  - Improved dynamic step size adaptation
  - Routability optimization
    - Simple but effective metal layer-aware superlinear cell inflation technique
- Superior solution quality for range of problem types
  - Standard cell placement: average HPWL reduction of 2.00% over best previous ISPD benchmark results
  - Mixed-size placement: average HPWL reduction of 2.73% over the best previous MMS benchmark results
  - Routability-driven placement: average 8.50% to 9.59% scaled HPWL reduction over previous leading academic placers for DAC-2012 and ICCAD-2012 benchmark suites

Notes
- Formulation is here (calculate demand / capacity) and here
- Dynamic step size updating is here
- Entry point for routability-driven is here
- Calculation of cell inflation for routability-driven placement is here and here

Note: WA vs. B2B: Same Target, Different Math

- OpenROAD / RePlAce has no global "weight matrix"
- Solver iteratively updates **gradients** of wirelength and density function
- "Weights" mentioned in code = "WA" = Weighted-Average weights, which are calculated for each pin in a net
  - At each iteration, weights are updated based on current pin positions
- Some facts
  - **Both B2B and WA are trying to capture HPWL-like wirelength**
  - **Both separate x- and y-contributions, then sum them**
  - **B2B** gives exact HPWL representation inside a quadratic formulation
  - **WA** gives a smooth approximation to max and min, controlled by smoothing parameter γ
  - **WA is differentiable and fits analytic nonlinear optimization**
- **Underlying challenge: How to optimize WL when HPWL is nonsmooth?**
  - B2B: Change objective into quadratic form that a quadratic placer can solve efficiently
  - WA: Smooth max and min operators so nonlinear placer can use gradients everywhere

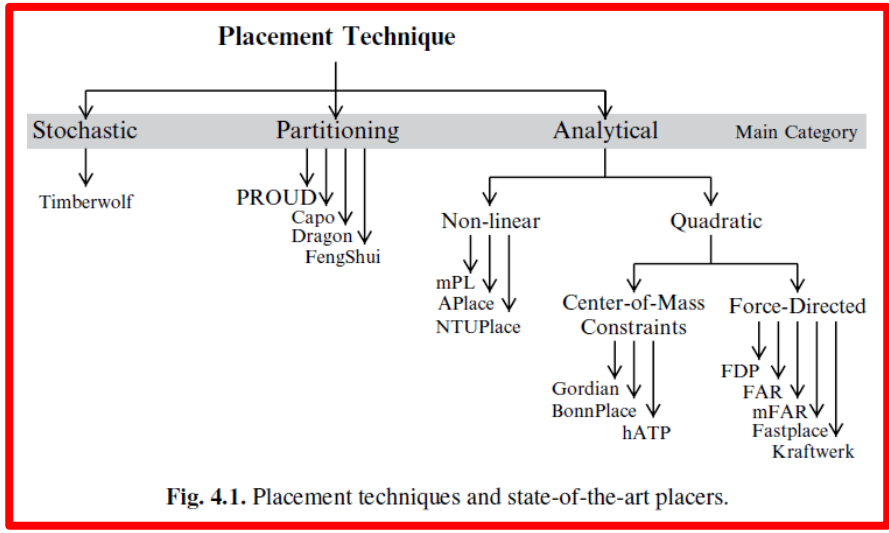
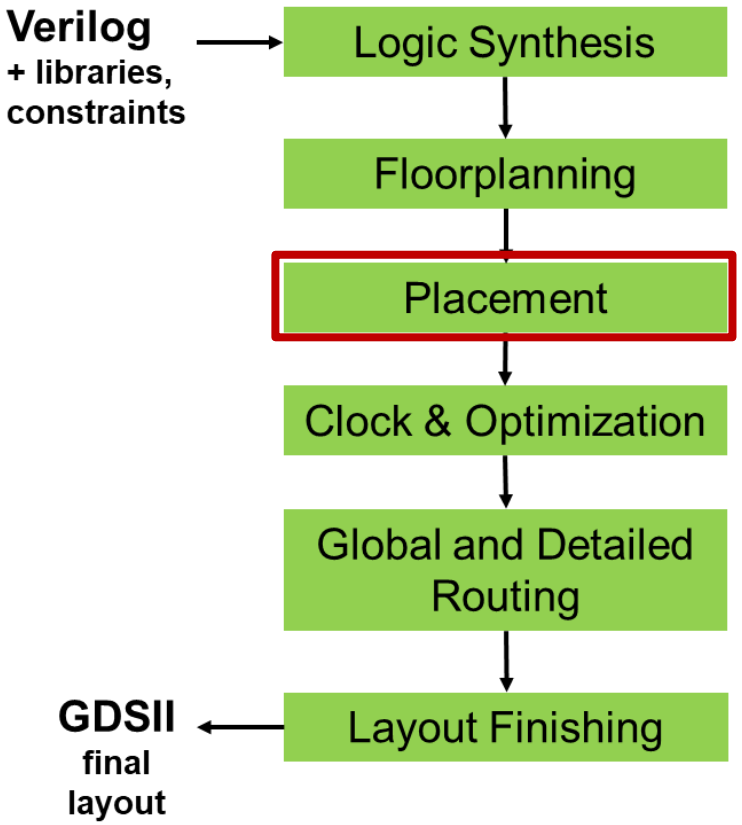
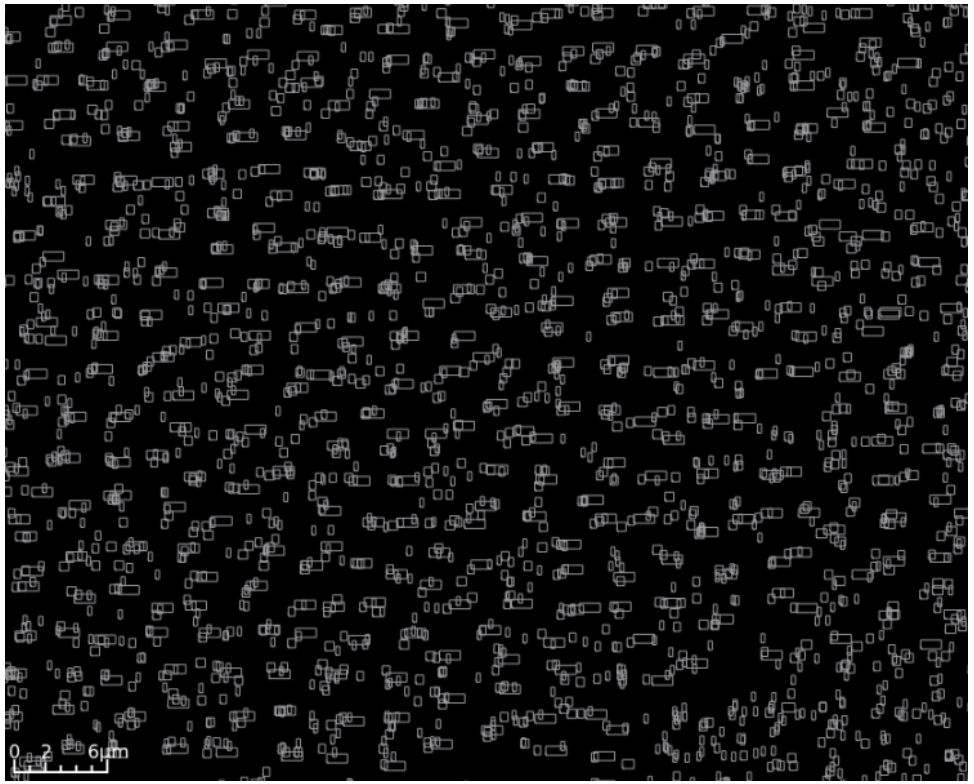


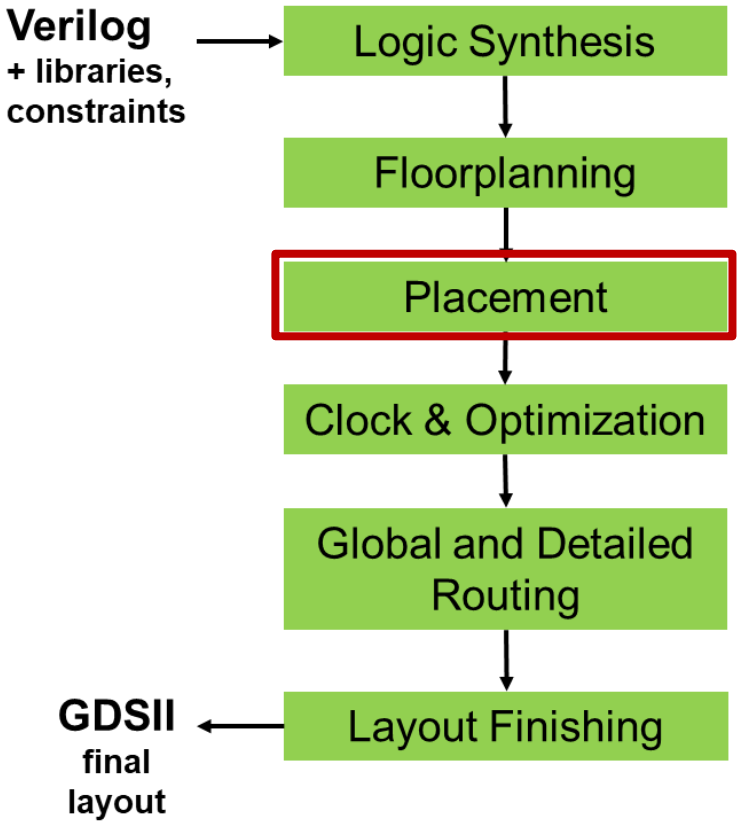
Fig. 4.1. Placement techniques and state-of-the-art placers.

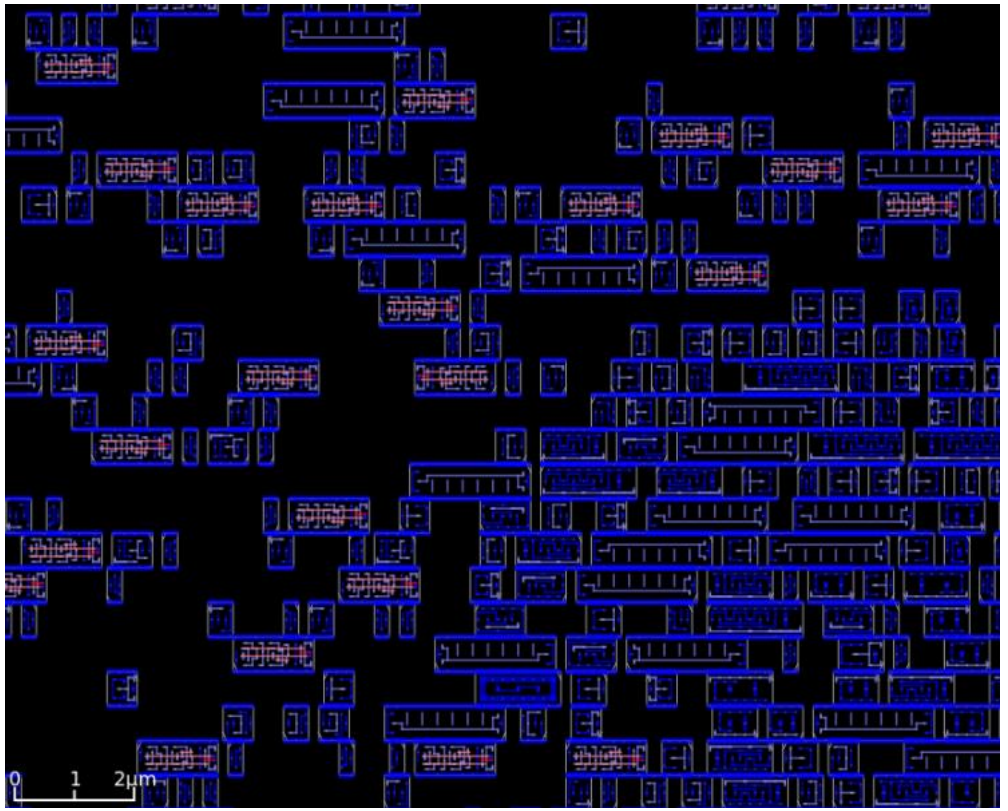
Global Placement Zoom-In





(Legalized) Detailed Placement Zoom-In

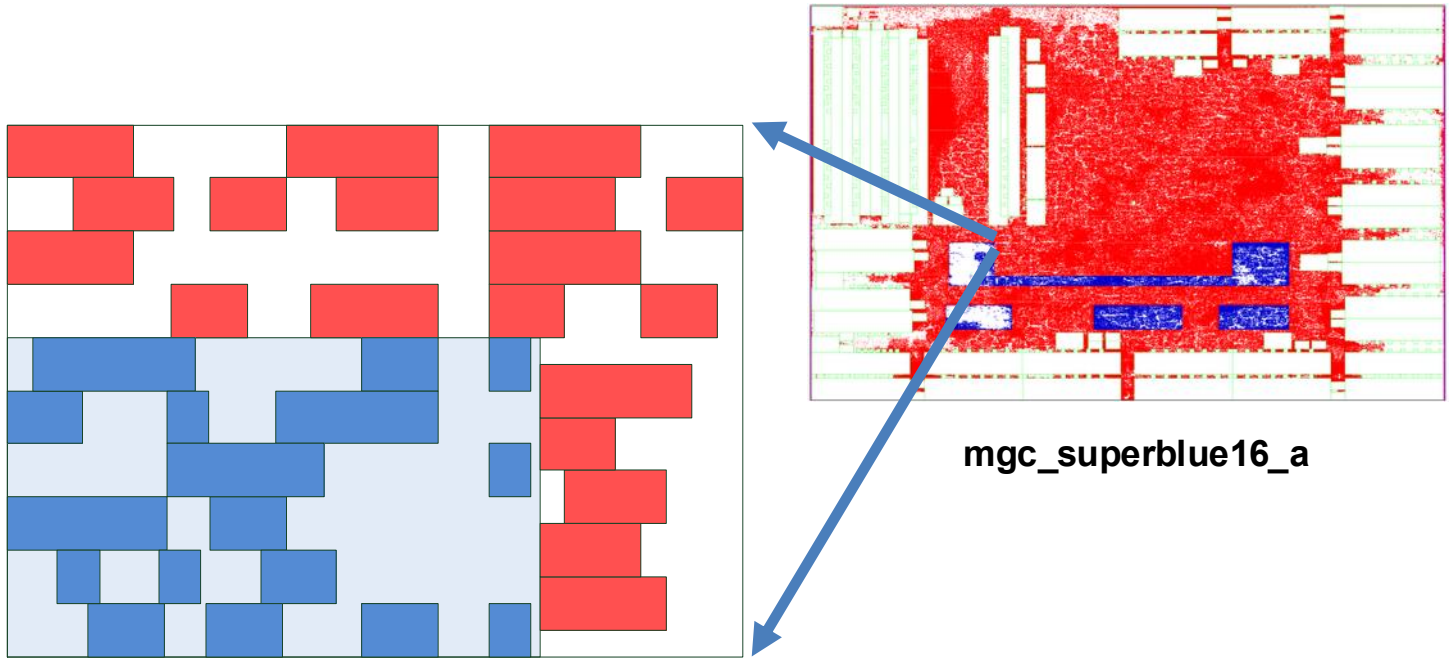




What did the tool do just now?
= legalization and “dpl”

Region constraints

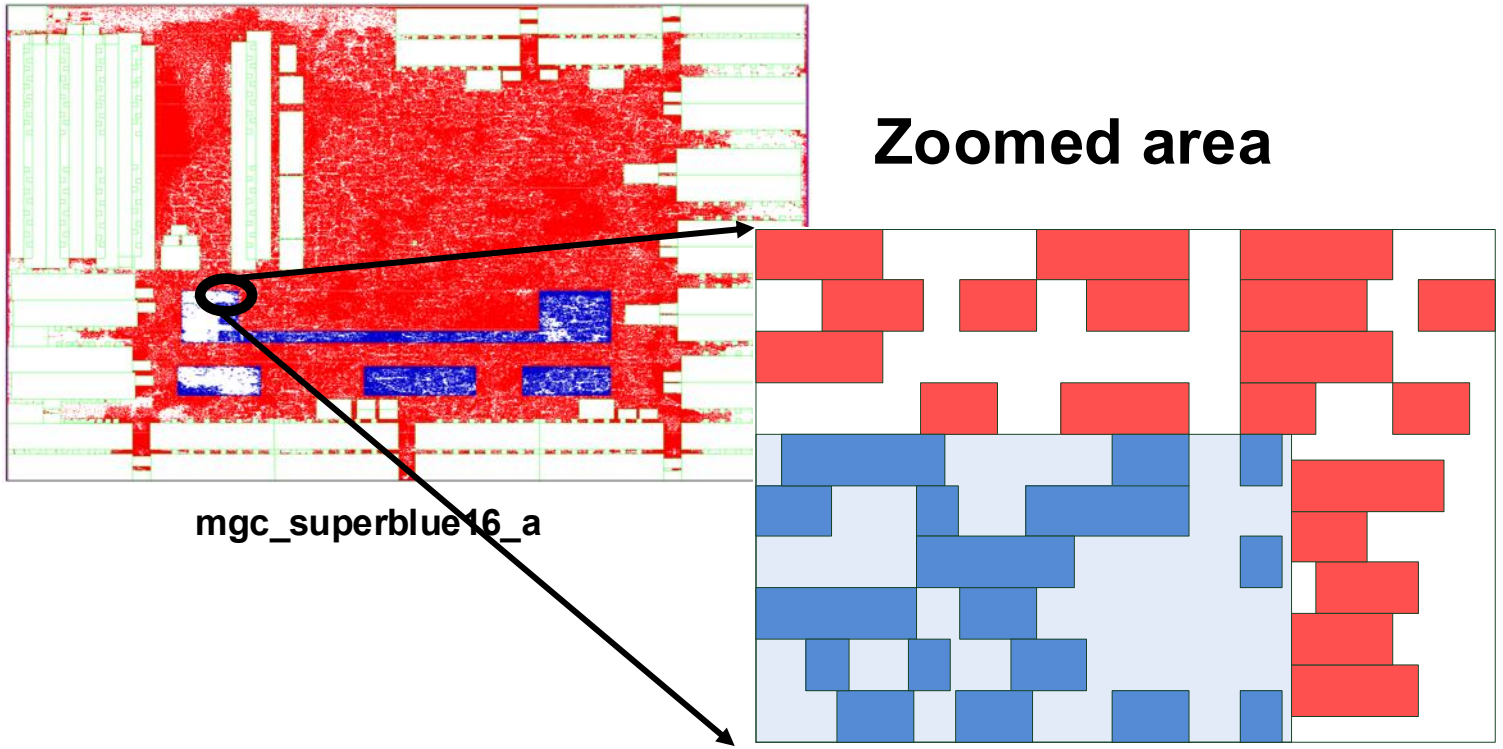
- Cells assigned to a region have to be placed inside the boundary of the region.



Zoomed area containing a region constraint

Fence region constraints

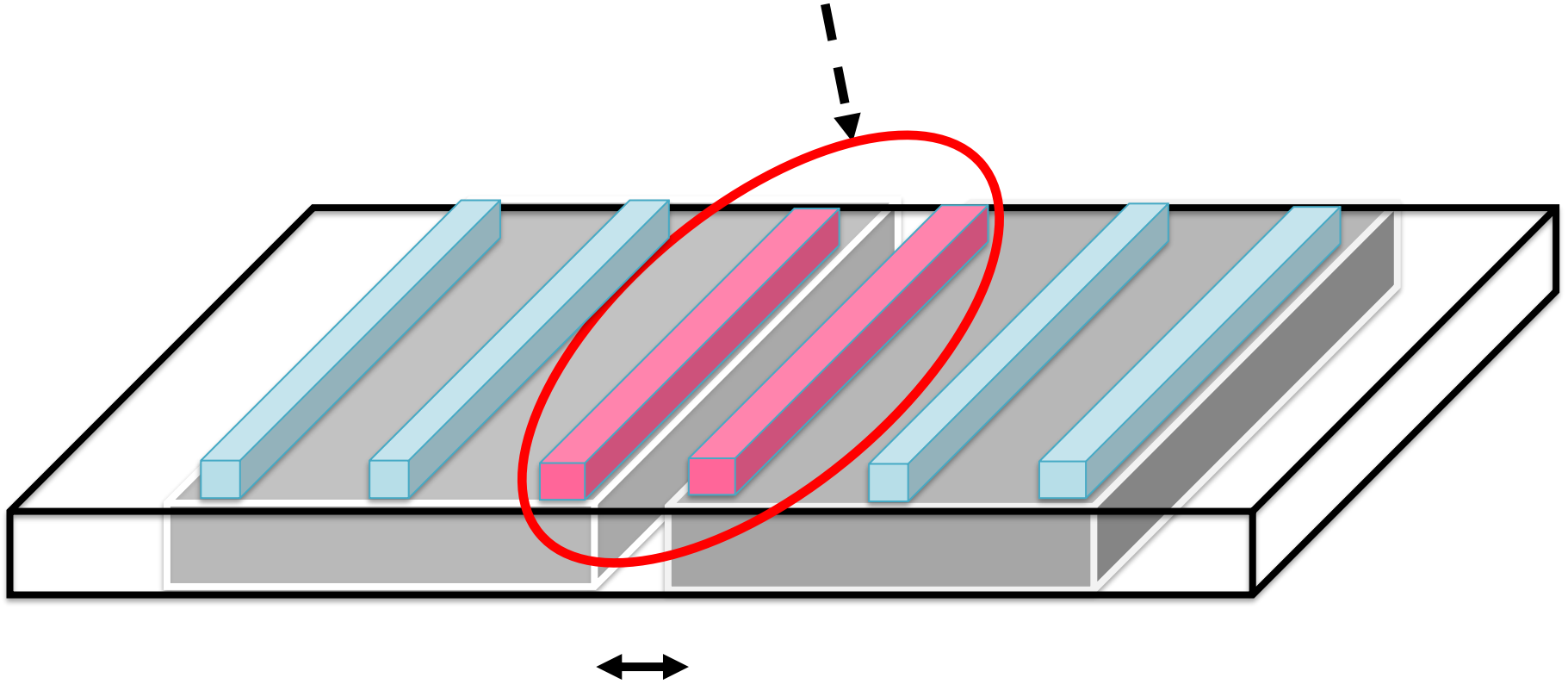
- Cells assigned to a fence region have to be placed inside the boundary of the region while other cells need to be placed outside.



Thanks: Nima Darav, AMD

Edge spacing constraints

Prone to pin access and short problems



Two cells are too close to each other

Thanks: Nima Darav, AMD

DPL

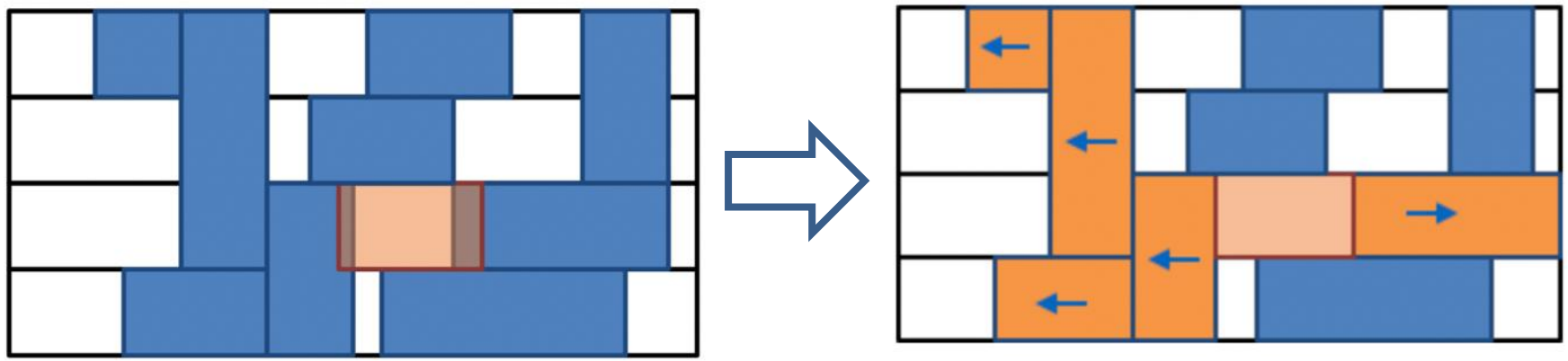
Legalization and detailed placement

- Global placement must be legalized
  - Cell locations typically do not align with power rails
  - Small cell overlaps due to incremental changes, such as cell resizing or buffer insertion
- Legalization seeks to find legal, non-overlapping placements for all placeable modules
- Legalization can be improved by detailed placement techniques, such as
  - Swapping neighboring cells to reduce wirelength
  - Sliding cells to unused space
- Software implementations of legalization and detailed placement are often **bundled** (dpl)

Thanks: Sang-Gi Do, Samsung

Mixed-height standard cell legalization

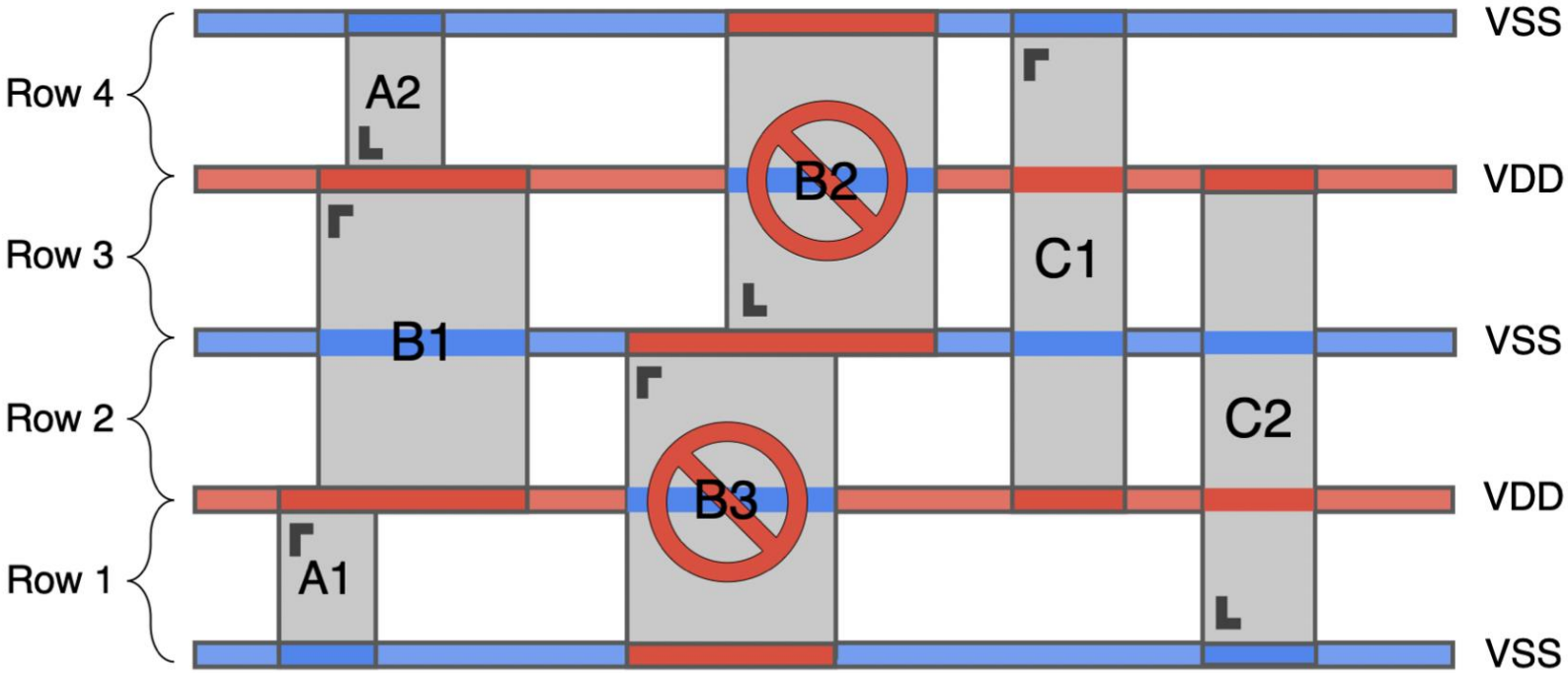
- **Legalization:** seek **legal** placement + remove **overlaps**



- **Mixed-height** standard cells: standard cells that span different row heights ← single/double/triple/quadruple
- **Mixed-height** standard cell legalization is **challenging**
  - Potential overlaps in **vertical** (+ horizontal) directions
  - Horizontal movement of a multi-height cell disrupts placement across all **rows** it overlaps
- Presence of fence regions aggravate the problem

Thanks: Sang-Gi Do, Samsung

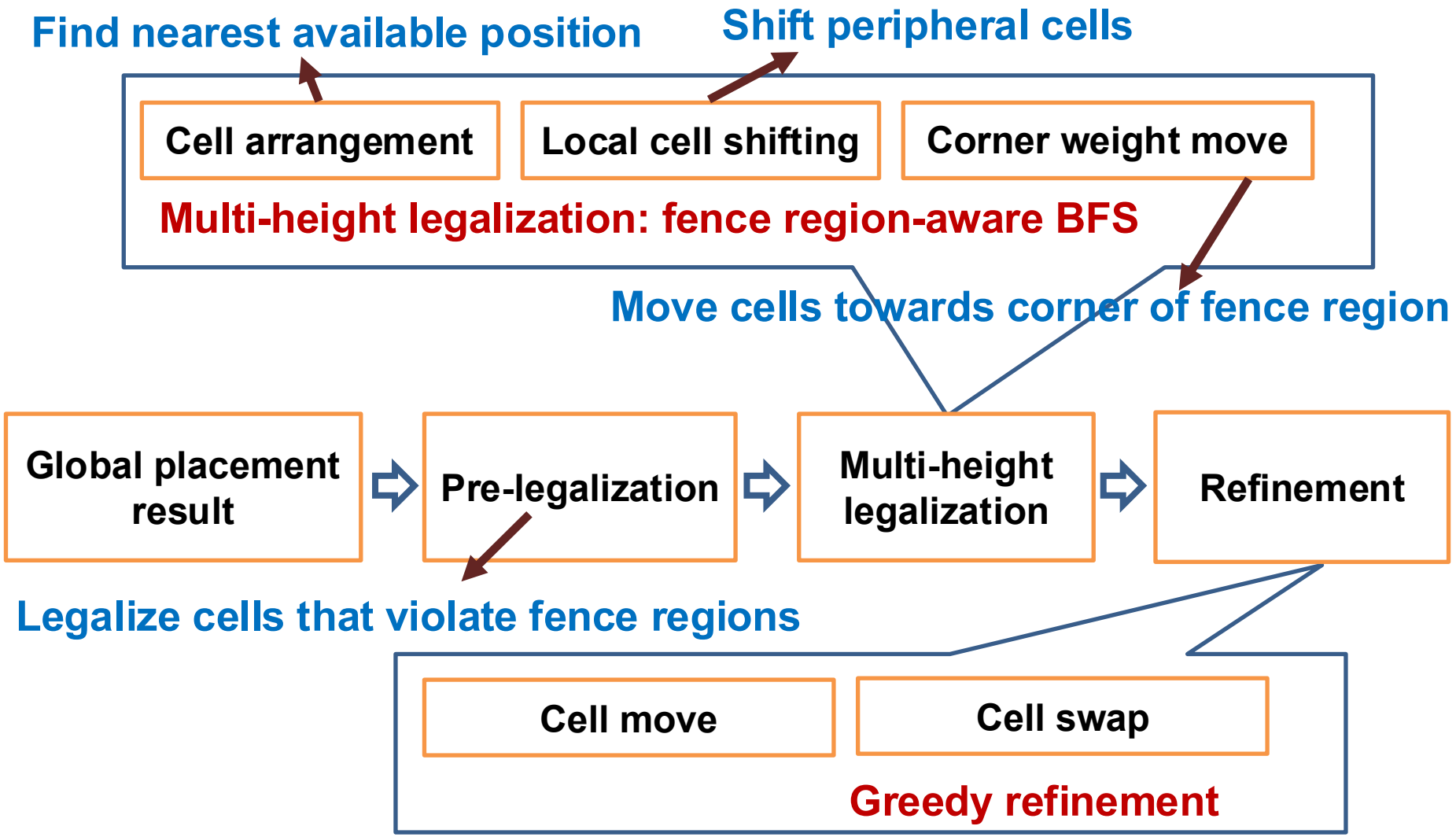
An example of power rails violation



- Red bars: power (VDD); blue bars: ground (VSS)
- A1, A2: single height standard cells; B1, B2: double height standard cells; C1, C2: triple height standard cells
- B2 and B3 **violate power rails alignment**

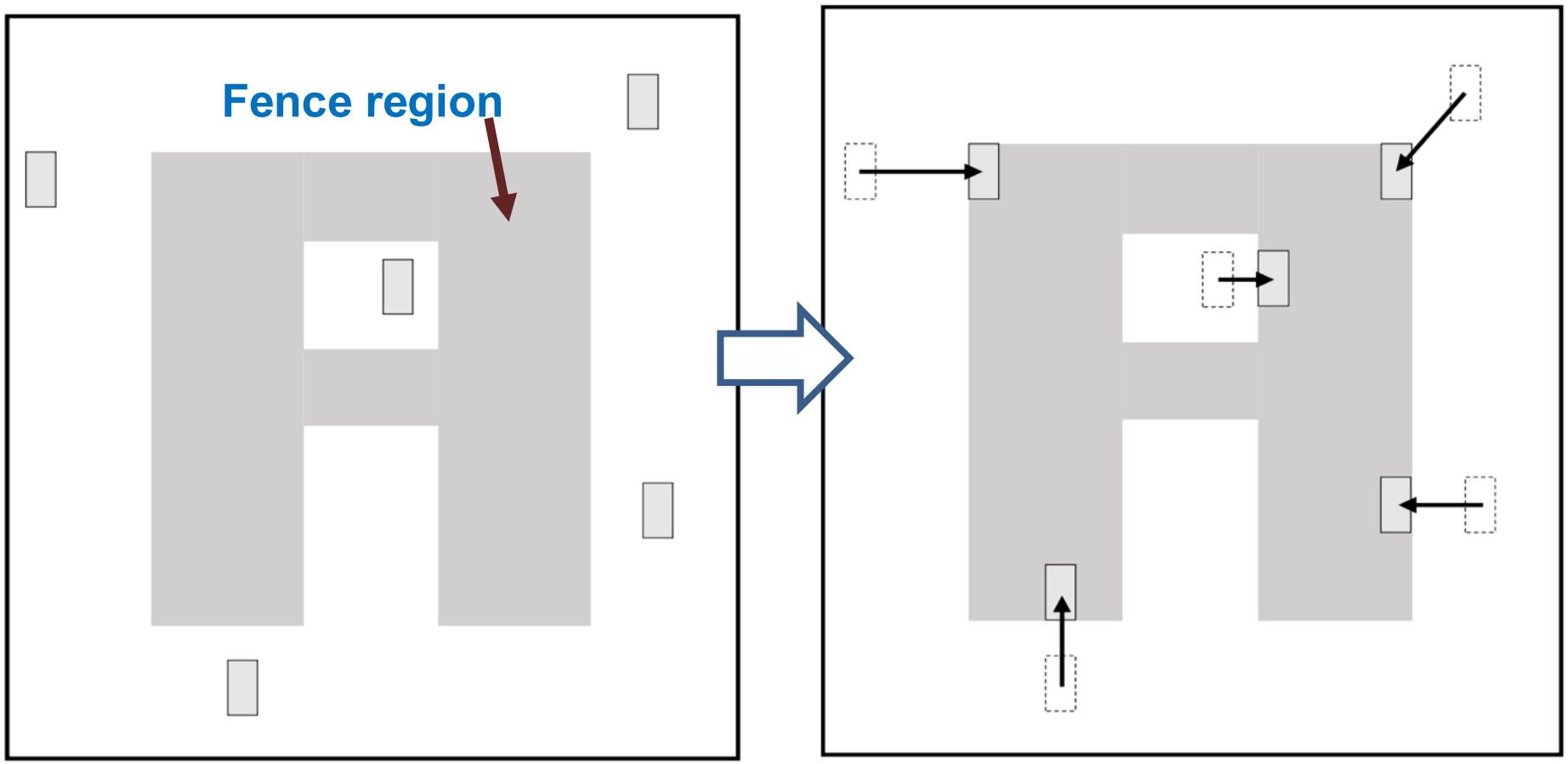
Overall framework in dpl

- Fence region-aware (FRA) BFS algorithm (paper)



Pre-legalization – an example

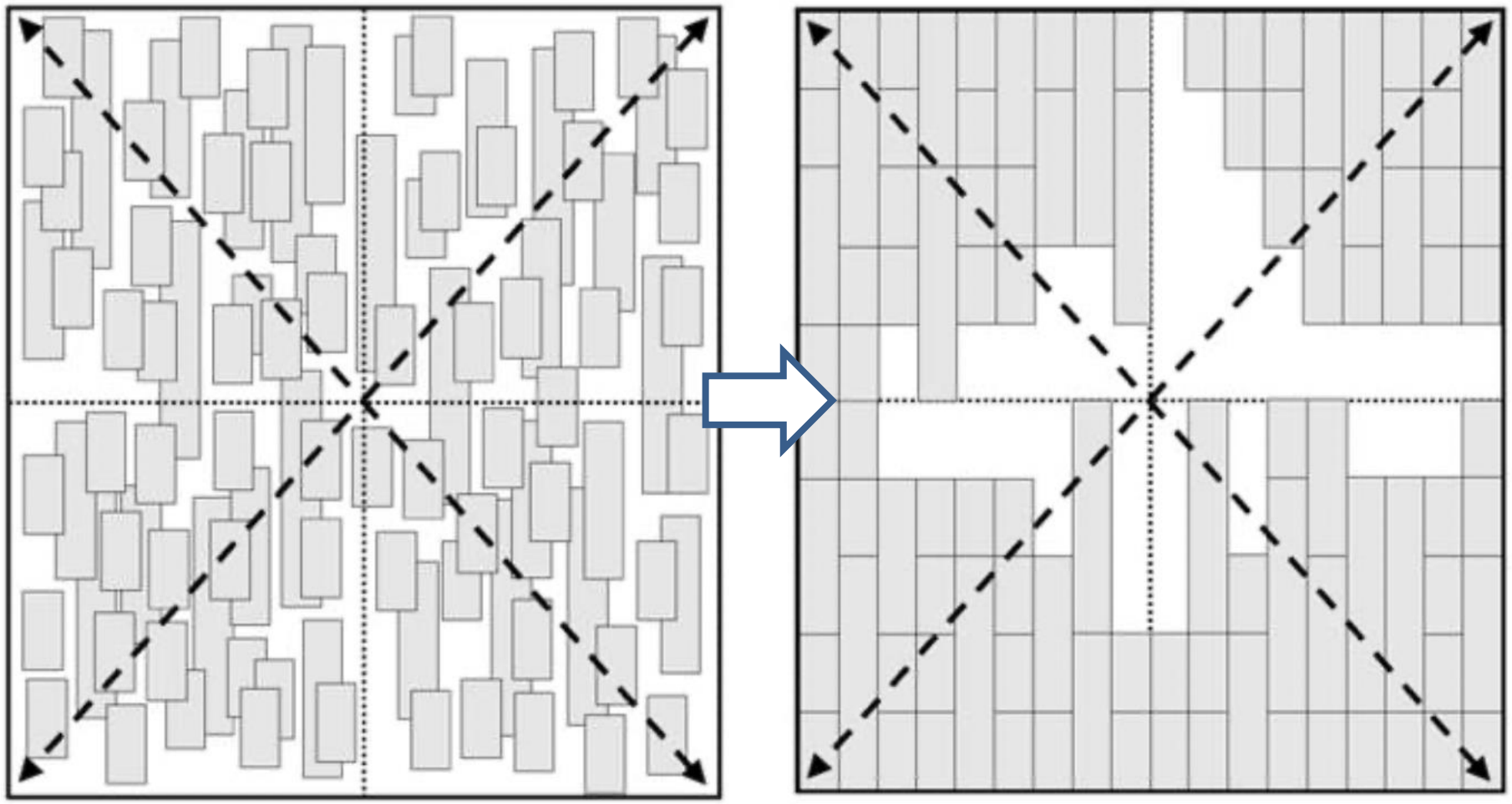
- Grey cells outside fence region are pushed into the fence region



Thanks: Sang-Gi Do, Samsung

Corner weight move – an example

- Push standard cells towards the corners of the fence region

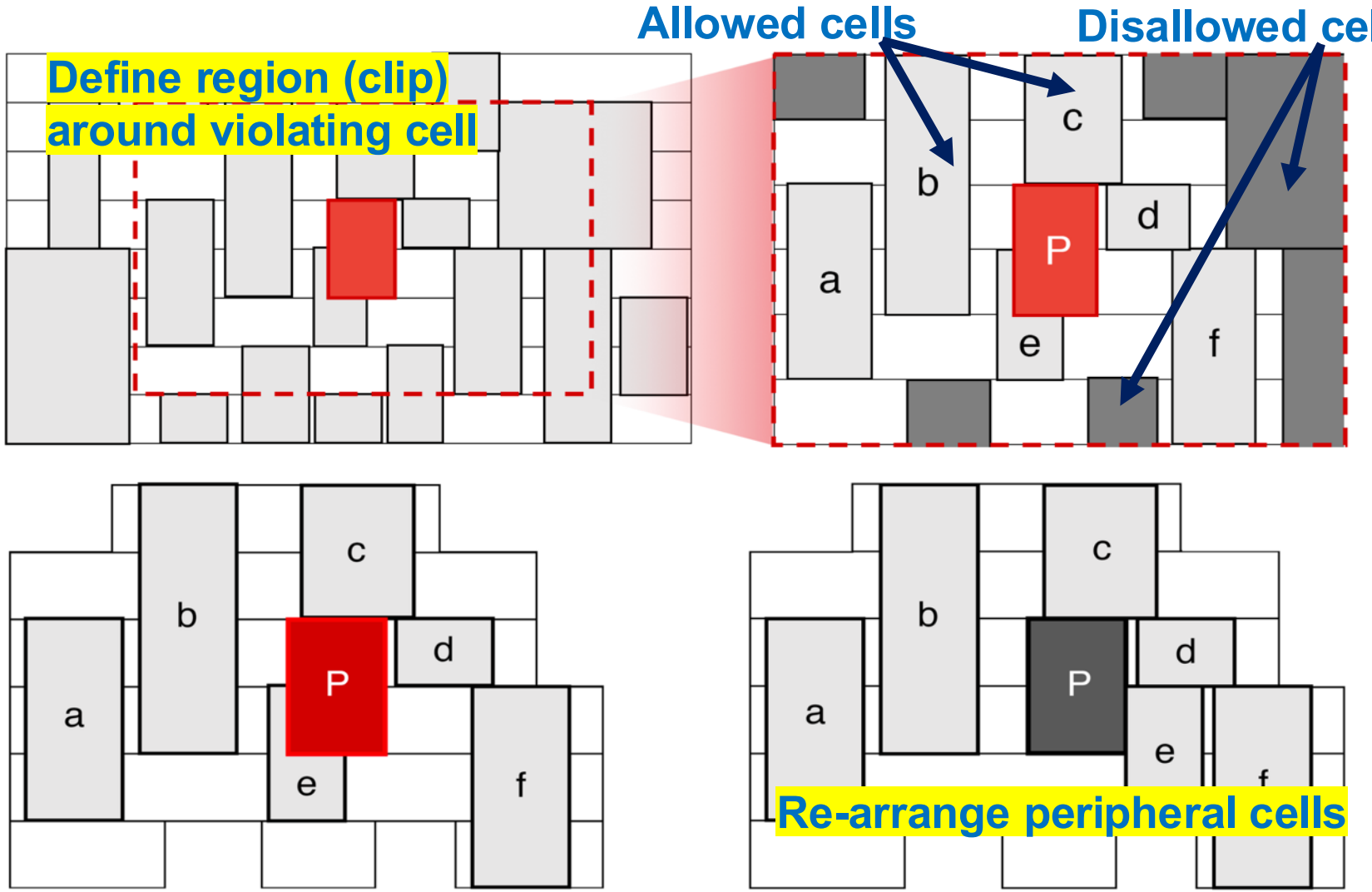


Before corner weight moves

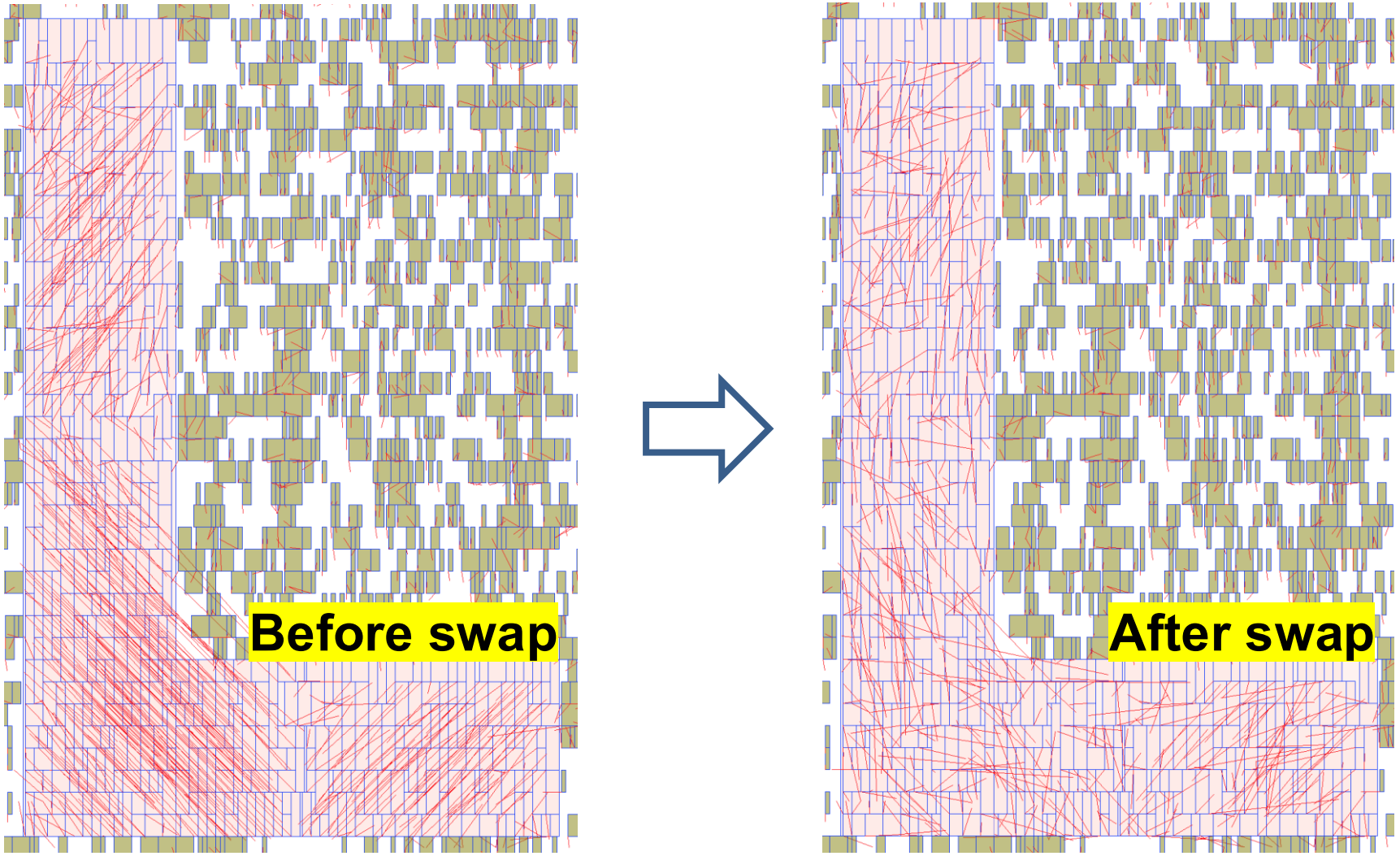
After corner weight moves

Local shifting – an example

- Set a region around violating cell + rearrange peripheral cells to resolve violation



Cell swaps – an example



Benchmark : des_perf_b_md2. Cell count : 112644.
Max cell row : 4 Fence Utilization : 96.2%

the old “dpo” which is now part of “dpl” …

DPO in a nutshell (1/2)

- DPO runs a detailed placement "script" that can perform a sequence of different optimization algorithms (see Optdp.cpp).
- Implemented algorithms (variations of what's been described in literature as there are many incarnations of the different algorithms): Maximum independent set matching [1], Optimal reordering [2], Median improvement (global and vertical swapping) [3], greedy improvement [4]
- Sample script string:

algorithm to run (e.g., gs = global swap, ro = optimal reordering, etc.)

```
dtParams.script_ = "mis -p 10 -t 0.005' gs -p 10 -t 0.005 ; vs -p 10 -t 0.005; ro -p 10 -t 0.005; default -p 5 -f 20 -gen rng -obj hpwl -cost (hpwl)"
Dpo::Detailed dt(dtParams);
dt.improve(mgr);
```

runs the script

algorithm-specific parameters

Thanks: Andrew Kennings, Altera

DPO in a nutshell (2/2)

- Most algorithms can optimize hpwl or displacement (other objectives such as timing, congestion, etc.) are not well accounted for which is a “consequence” of the algorithm.

- The greedy algorithm is actually very flexible; it uses the idea of “move generators” and “cost objects” that can propose any set of cell movements (cells assigned to new locations) which are then evaluated by the “cost objects” to form a cost function.
  - Proposed moves are accepted or rejected based on cost and move generators and cost objects track the current state of the placement (for incremental computation).
  - Need to be careful to support the required object pieces to add something; e.g., routines need accept() and reject() methods.

- DPO is largely not great with multi-height cells
  - E.g., greedy algorithm can only do simple swaps and moves and is not good at “shifting other cells out of the way”.

Thanks: Andrew Kennings, Altera

Some References

1. K. Doll, F. M. Johannes, and K. J. Antreich. “Iterative placement improvement by network flow methods”, IEEE Trans. Computer-Aided Design of Integrated Circuits and Systems, 13(10):1189-1200, 1994
2. A. E. Caldwell, A. B. Kahng and I. L. Markov, “Optimal partitioners and end-case placers for standard-cell layout”, *Proc. ACM Intl. Symp. on Physical Design*, April 1999, pp. 90-96.
3. Min Pan, Natarajan Viswanathan and Chris Chu, “An efficient and effective detailed placement algorithm”, IEEE/ACM International Conference on Computer-Aided Design, pages 48-55, 2005.
4. Andrew A. Kennings, Nima Karimpour Darav, Laleh Behjat “Detailed placement accounting for technology constraints VLSI-SoC 2014:1-6

BACKUP